

PetSim RPG: A Virtual Pet Simulation and Training System



**A Final Project Presented to
the Faculty of the Institute of Computing,
Davao del Norte State College,
New Visayas, Panabo City**

Dumail, Mike Chester

Genite, John Mark

Batoon, Jean

Novicio, Mark Lloyd

December 2025

Table of Contents

CHAPTER 1

INTRODUCTION

Background of the System.....	1
Objectives.....	2
Flowchart.....	3
Pseudocode.....	17
Code.....	27

CHAPTER II

RESULT AND DISCUSSION

Result.....	42
Discussion.....	48

CHAPTER III

CONCLUSION AND RECOMMENDATION

Conclusion.....	49
Recommendation.....	49

REFERENCES.....	51
------------------------	-----------

CHAPTER I

INTRODUCTION

A. Background of the System

In recent years, educational and simulation games have gained attention for their ability to foster responsibility, problem-solving, and emotional awareness among learners. Among those games, virtual pet systems highlight caring for a pet in ways that mirror real life. The proposed system is an RPG console-based virtual pet game where players adopt a single pet with the main goal of training it to reach level 10. Fundamental mechanics include mood and energy, which interact directly with growth, levels, and the EXP system. This ensures that the system emphasizes structured progress simulating energy and mood management, planning, and decision-making. Studies show that participants who actively fed, played with, and cared for virtual pets reported greater attentiveness, satisfaction, and emotional well-being compared to passive activities such as watching animal videos [1]–[3]. Virtual pets also provide a sustainable way to teach life skills to individuals who cannot yet take on the responsibility of owning a real pet, while encouraging empathy and critical thinking through interactive choices [4].

The design of the virtual pet system emphasizes balanced simplicity with engagement, ensuring the user's focus on progression without overcomplicated mechanics. Core features such as mood, growth, energy, leveling, and EXP establish clear goals and reinforce cause-and-effect learning based on user decisions. Programs for emotional well-being that are game-based have been shown to positively affect classroom atmosphere and student behavior [5]. Likewise, gamified learning increases motivation, engagement, and active participation, while immediate feedback and clear progression paths enhance learning outcomes [6],[7]. Interactive learning activities with virtual pets further encourage empathy and responsibility by simulating real-life animal care [8], and virtual pet therapy apps have demonstrated improvements in attentiveness and satisfaction [9].

Beyond single-player progression, the system introduces a multi-user feature that allows each participant to manage their own pet. This reinforces personal responsibility and engagement, ensuring individualized journeys that differ from other users. Such personalization aligns with gamification principles, where structured progression and feedback enhance motivation and learning outcomes [10],[11]. By combining individualized pet care with collaborative learning environments, the system not only entertains but also cultivates responsibility, emotional awareness, and cognitive growth.

B. Objectives

The goal of this program is to create a Multi-User Console-Based RPG Virtual Pet Simulation System that embodies the basic CRUD operations. Specifically, it aims to:

1. Allow the user to input signup and login credentials (username and password).
2. Allow the user to adopt (Create) and select a pet of their own choice and assigning:
 - a. Pet type (Cat, Dog, Duck, Pig, Rabbit, Horse)
 - b. Pet name
3. Allow the user to View (Read) the pet's current information and status.
4. Allow the user to Interact with the pet through actions:
 - a. Training
 - b. Playing
 - c. Feeding
 - d. Resting
5. Track and change (Update) the pet's status after each user action that includes:
 - a. Energy
 - b. Mood
 - c. Growth Stage
 - d. Level and EXP
6. Allow the user to release (Delete) its pet.
7. Implement the multi-user feature through the use of parallel array storage to ensure individualized progression.

C. Flowchart

Game Loop Process

Figure 1 shows the game loop process of the system. It begins with the initialization of the game variables, preparing for the game system execution. The system then checks whether the game is still running. If the condition is true, the program continues to execute and loops back to the running state, maintaining gameplay. Otherwise, the system prints a farewell message and terminates the process.

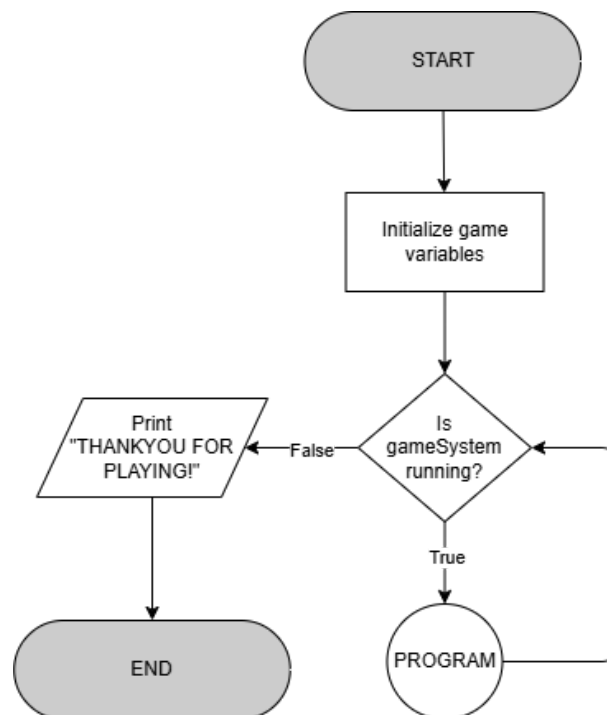


Figure 1. Game Loop Process

Access Menu Process

Figure 2 illustrates the authentication decision process of the system. It begins by checking whether the user has not yet logged in. If the user is unauthenticated, the system presents options to sign up, log in, or exit. The user's input is then validated to ensure it is an integer and falls within the acceptable range. If the input is invalid at any stage, the system prints an error message and prompts the user to try again. This loop continues until a valid input is received, ensuring proper navigation and secure access to the system.

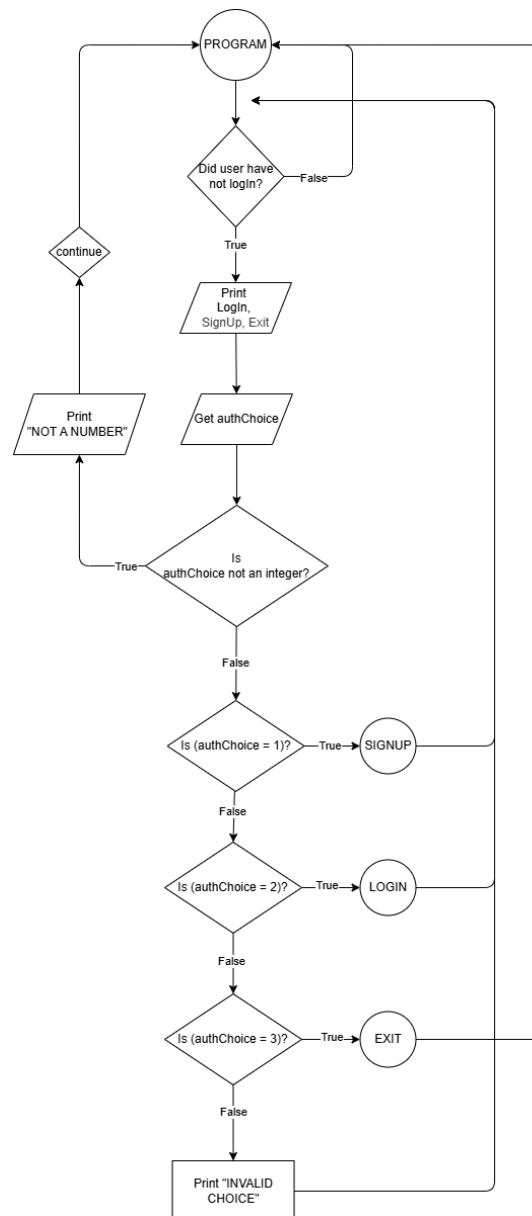


Figure 2. Access Menu Process

User Signup Process

Figure 3 shows the user signup validation process. It begins by checking if the system can accept more users. If so, the user enters a username, and the system checks for duplicates. If a duplicate is found, an error message is shown and goes back to the Access Menu (Figure 2). Otherwise, the user sets a password, and the account is created successfully.

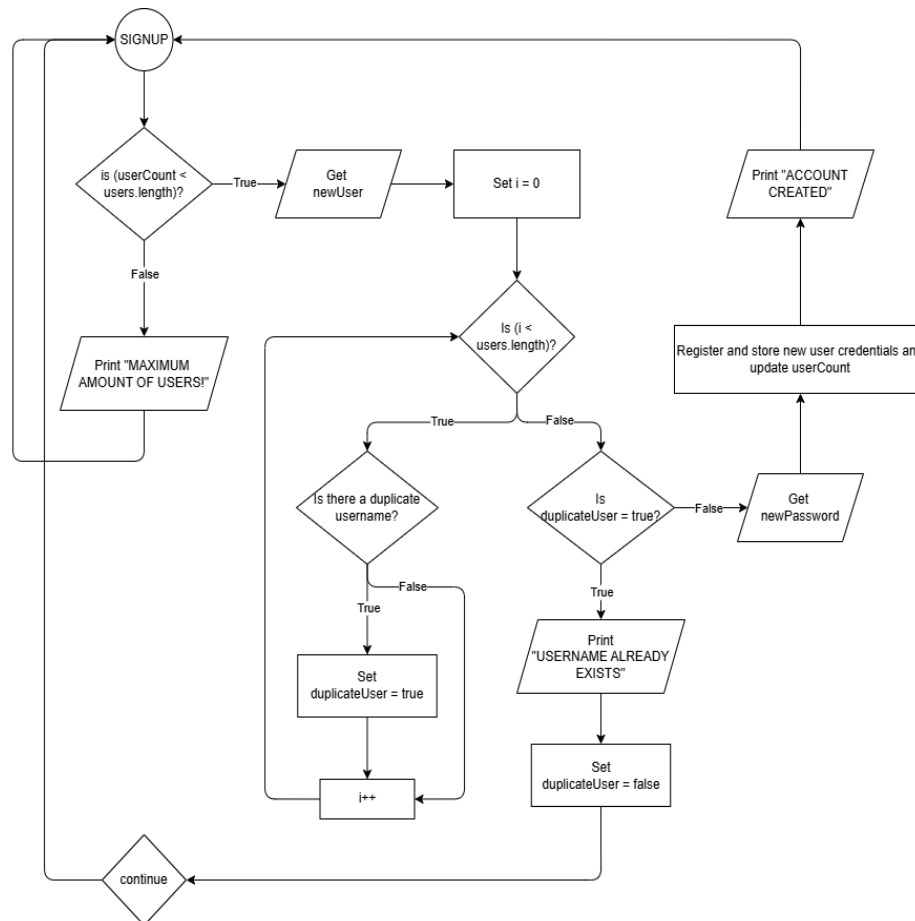


Figure 3. User Signup Process

User Login Process

Figure 4 shows the user login validation process. It begins by checking if any user accounts exist. If none are found, the system prompts the user to create an account. Otherwise, the user enters their credentials, and the system loops through stored accounts to verify them. If a match is found, login is successful and the game begins. If credentials are incorrect, an error message is displayed.

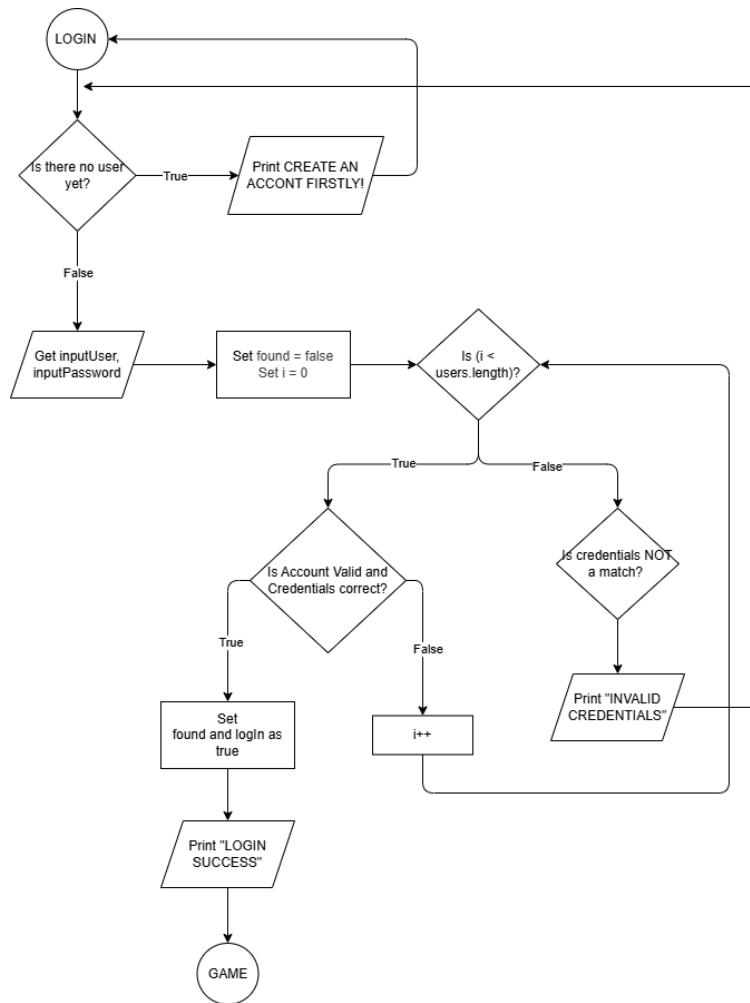


Figure 4. User Login Process

User Exit Process

Figure 5 shows the user exit process of the system. It begins by setting all system states to false, preparing the program for termination. The system then prints an exit message to the user. Finally, a break statement is executed to exit the loop and complete the process.

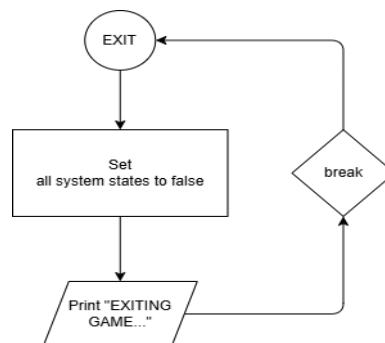


Figure 5. User Exit Process

PetSim RPG Menu Process

Figure 6 shows the main menu process of PetSim RPG. It first checks if the user is logged in and hasn't picked any options. The system then prompts the user to choose an option and validates the input to ensure it is a number and an integer. If the input is valid and within range, the system routes the user to adopt, view, interact with, release a pet, or log out. Otherwise, it will print error messages and prompt the user to try again.

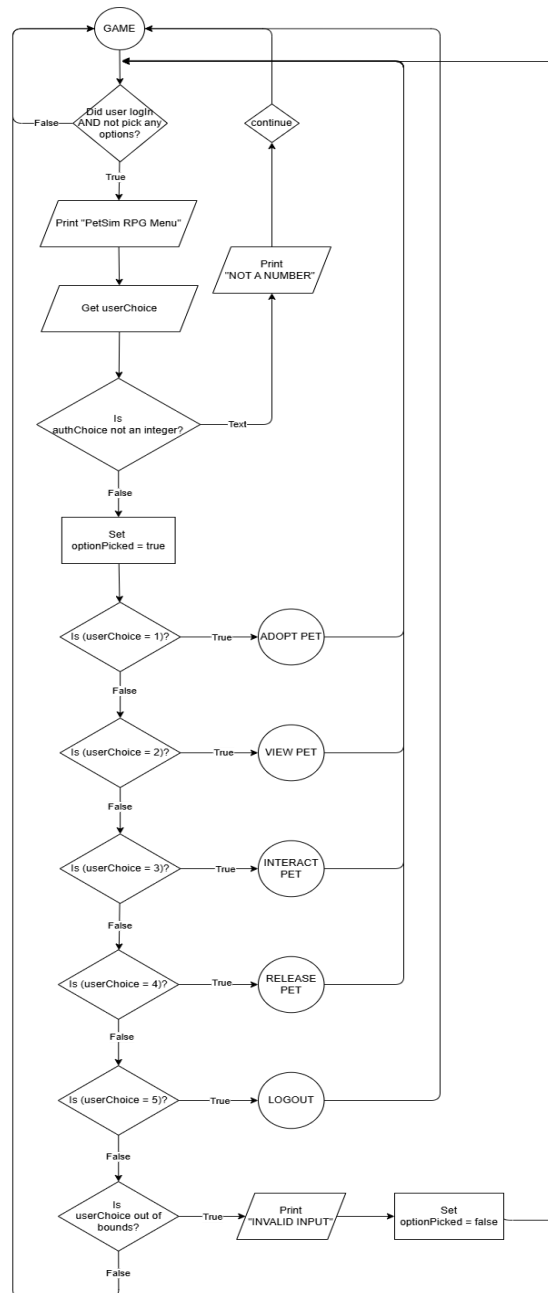


Figure 6. PetSim RPG Menu Process

Adopt Pet Process

Figure 7 illustrates the adopt pet process. It begins by checking whether the user already has a pet. If not, the system displays a list of pet types and prompts the user to select one by entering a number. Invalid inputs trigger error messages and repeat the prompt until a valid option is chosen. Once a pet type is selected, the user enters a name for the pet. The system then initializes the pet's starting stats and displays both the pet's name and type.

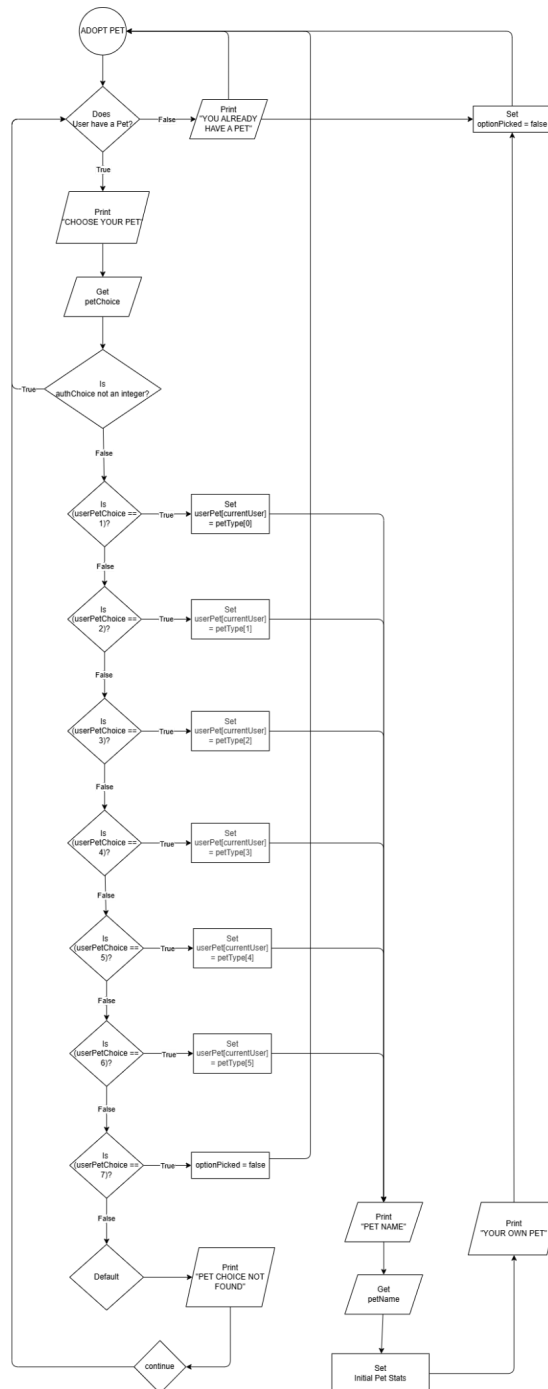


Figure 7. Adopt Pet Process

View Pet Process

Figure 8 illustrates the view pet process. The system checks if the user has a pet. If they do, it displays the pet's information. If not, it shows a message saying the pet is nowhere to be found. After either outcome, the system resets the picked option and returns back to the game menu (Figure 6).

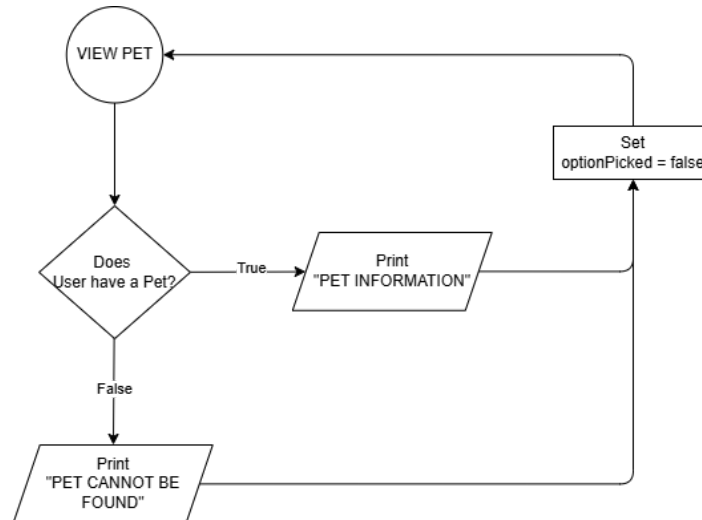


Figure 8. View Pet Process

Interact Pet Process

Figure 9 outlines the interact pet process. It begins by checking if the user has completed the main goal (reaching pet level 10). If so, the user is sent back to the game menu. If not, the system checks for an existing pet. If no pet is found, a message is displayed and the user returns to the game menu (Figure 6). Otherwise, the system presents interaction options and prompts for input. Invalid entries trigger error messages until a valid choice is made. After each interaction, the flow continues to the after-action process (Figure 16), then returns to the game menu.

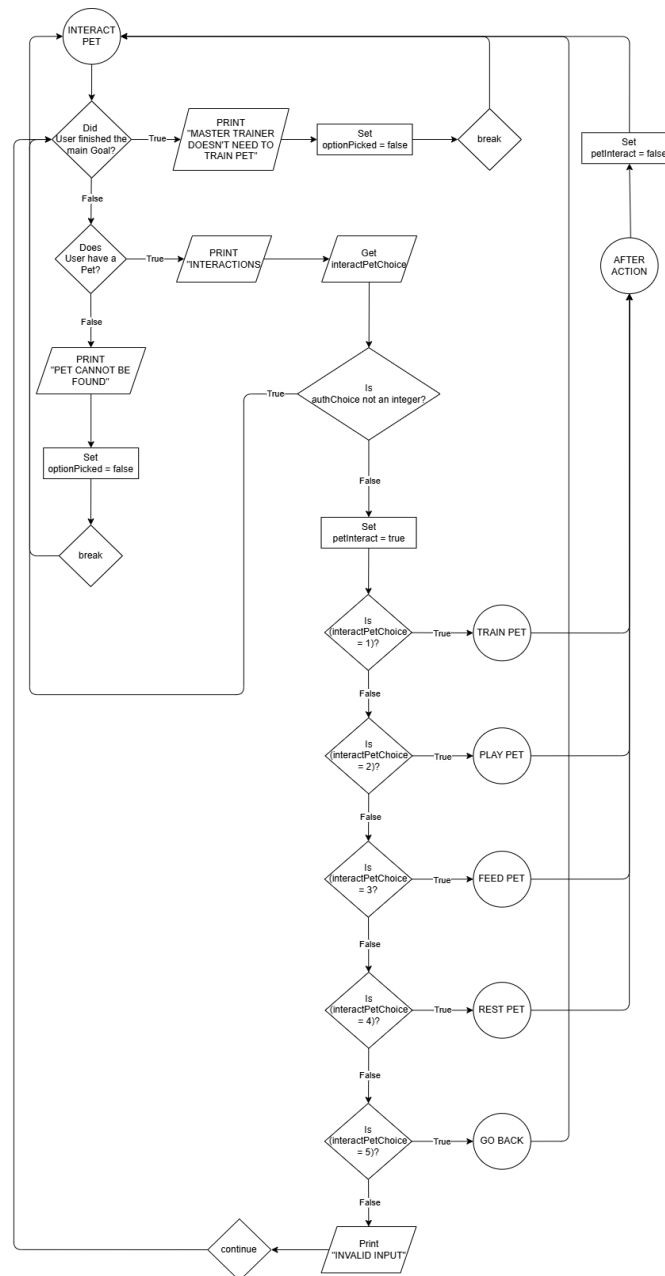


Figure 9. Interact Pet Process

Release Pet Process

Figure 10 illustrates the release pet process. It begins by checking if the user has a pet. If not, a message is displayed and the process ends. If a pet exists, the system asks for confirmation to release it. A valid “yes” input releases the pet and confirms the action. A valid “no” input cancels the release. Any other input triggers an error message. The process then resets and returns to the menu.

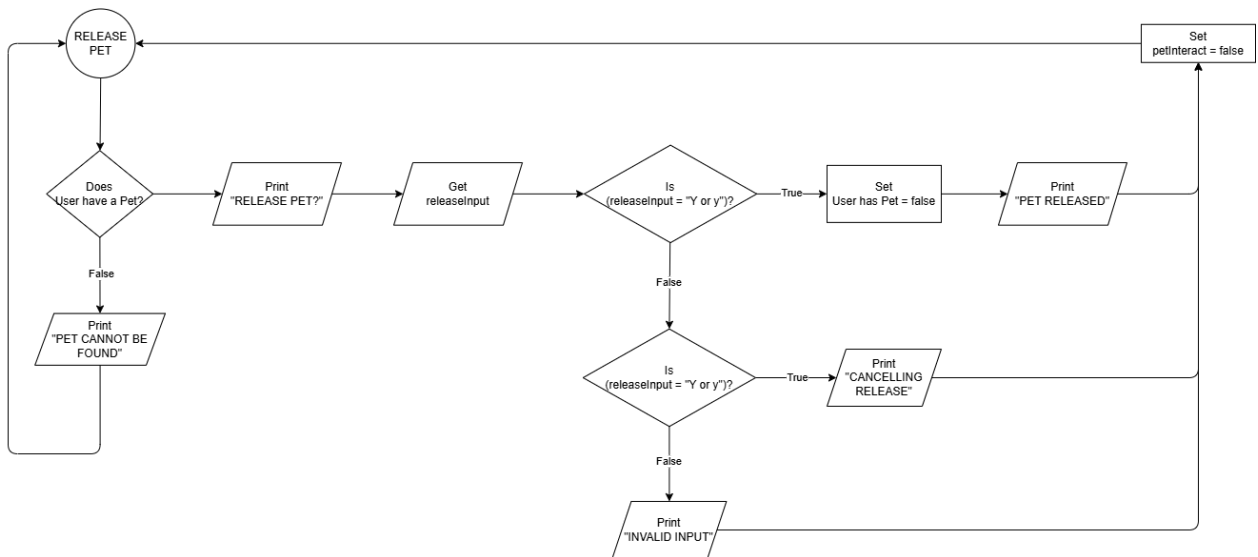


Figure 10. Release Pet Process

Logout Process

Figure 11 illustrates the logout process. The system displays a logout message, then resets the menu and login status. After completing these steps, it returns to the logout entry point going back to the game menu.

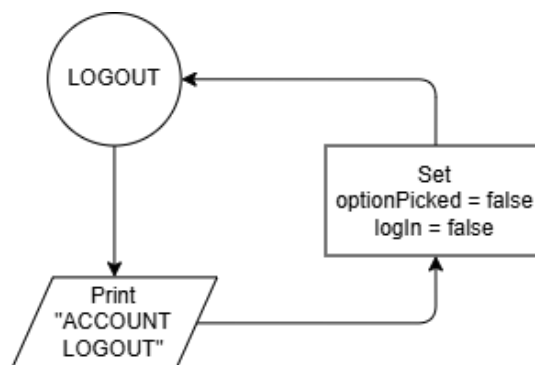


Figure 11. Logout Process

Train Pet Process

Figure 12 shows the train pet process. The system first checks if the pet has enough energy. If not, it prints a message and lowers the pet's mood. If energy is sufficient, it prints a training message, updates stats, and adds experience. It then checks the pet's mood: if it's negative, it resets to zero; if it's out of bounds, it's set to 3; otherwise, the mood remains unchanged.

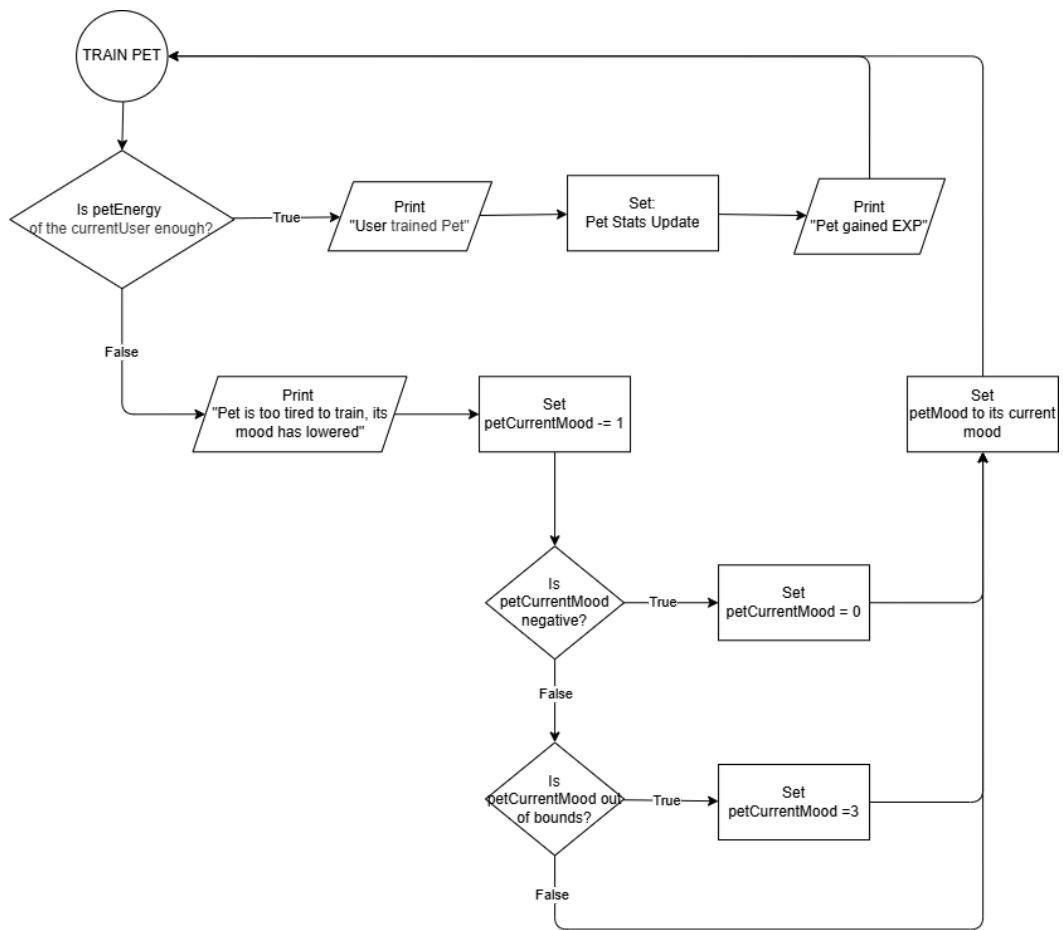


Figure 12. Train Pet Process

Play Pet Process

Figure 13 illustrates the play pet process. The system first checks if the pet has enough energy. If not, it prints a message and lowers the pet's mood. If energy is sufficient, it prints a play message, updates the pet's stats, and adds experience. It then checks the pet's mood: if it's negative, it resets to zero; if it's out of bounds, it's set to 3; otherwise, the mood remains unchanged.

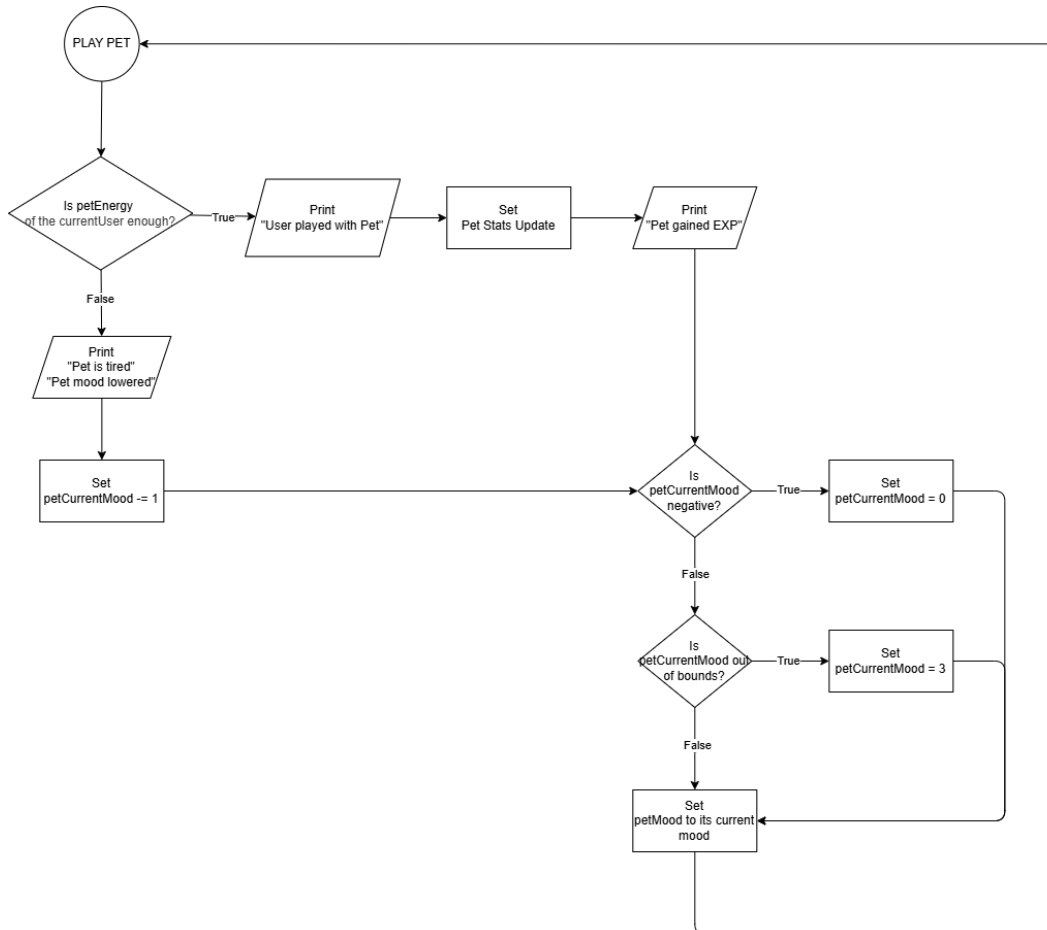


Figure 13. Play Pet Process

Feed Pet Process

Figure 14 illustrates the feed pet process. The system prints a message that the user feeds the pet, increasing the pet's energy by a random amount, and checks if the energy exceeds the limit. If it does, the energy is capped. Then it checks the pet's mood: if it's within bounds, the mood increases; if not, it's set to the highest level. Finally, the system prints a message confirming the pet gained energy.

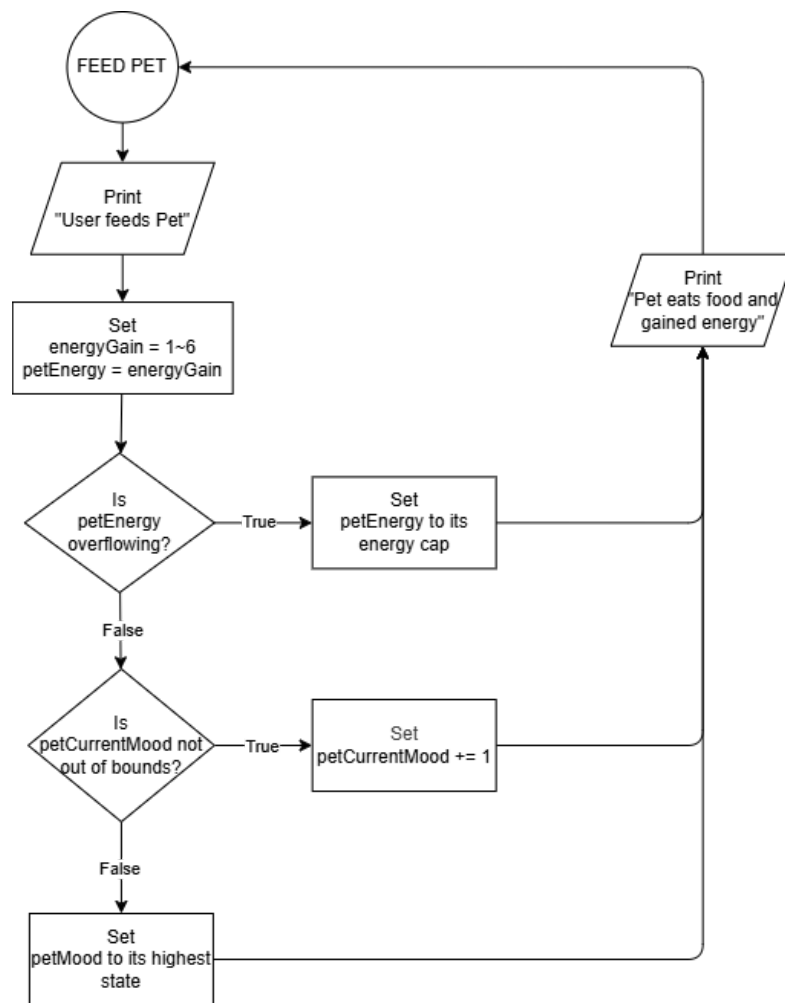


Figure 14. Feed Pet Process

Rest Pet Process

Figure 15 illustrates the rest pet process. The system prints a message indicating the pet is sleeping, then adds a random energy value between 1 and 10 (proportional to the pet's level) to the pet's current energy. If the new energy exceeds the maximum limit, it is capped. Lastly, a message confirms the energy gain of the pet before the process ends.

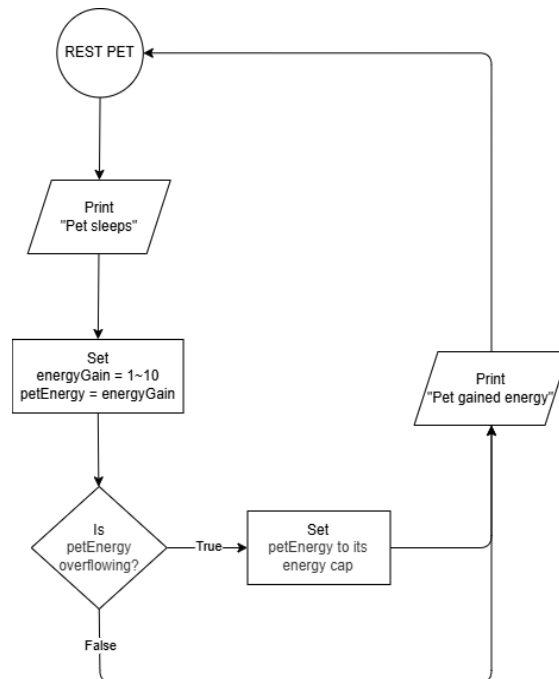


Figure 15. Rest Pet Process

Go Back Process

Figure 16 illustrates the exit process for the Interact Pet menu. It sets the option picked and pet interact into false and then goes back to the game menu after resetting those said flags.

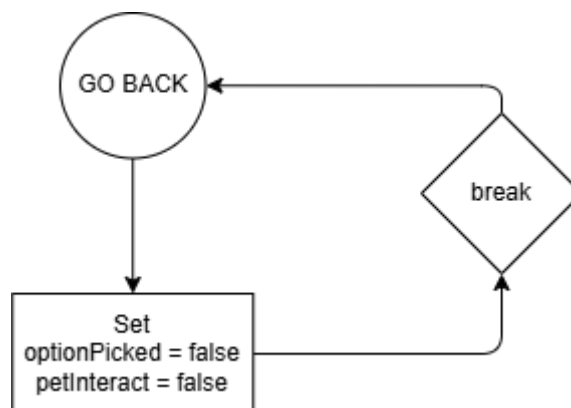


Figure 16. Go Back Process

After Action Process

Figure 17 illustrates the after action process. The system first checks if the user has a pet. If so, it evaluates whether the pet has gained enough experience to level up. If it does, the pet's stats are recalculated and a level-up message is displayed. If not, the pet continues to grow. The system then checks if the pet has reached the next growth stage or the final goal. If the goal is reached, final flags are set and a result message is printed. Otherwise, the process ends or displays the result based on progress.

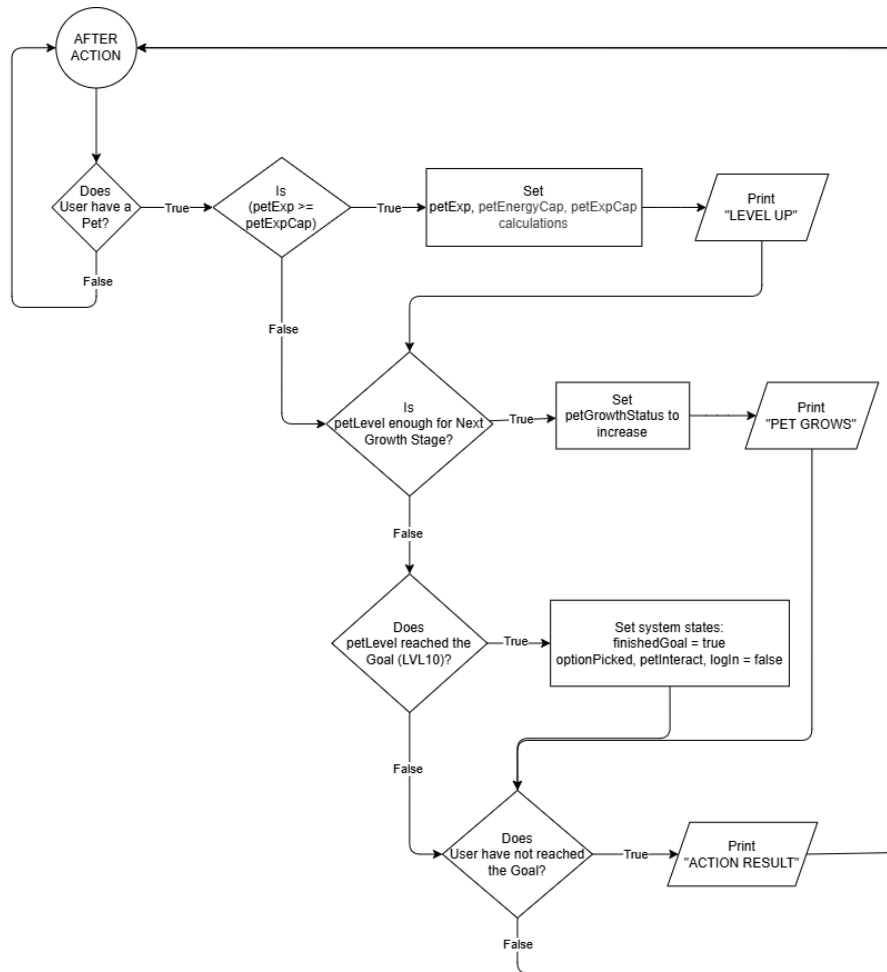


Figure 17. After Action Progress

D. Pseudocode

Game Loop

```
BEGIN GAME LOOP

    INITIALIZE VARIABLES
        SET capacity = 3    // maximum number of users

    USER VARIABLES
        users[capacity], passwords[capacity], hasPet[capacity], finishedGoal[capacity],
userPet[capacity]
        duplicateUser = FALSE
        currentUser = 0
        userCount = 0
        userID = 0

    PET VARIABLES
        petType = {"Cat", "Dog", "Duck", "Pig", "Rabbit", "Horse"}
        petDiffMoods = {"Sad", "Neutral", "Happy", "Joyful"}
        petDiffGrowth = {"Baby", "Child", "Teen", "Adult"}
        petGrowthStatus[capacity], petMood[capacity], petName[capacity]
        petExpCap[capacity], petExp[capacity], petLevel[capacity], petEnergy[capacity],
petEnergyCap[capacity]
        petCurrentMood = 0
        petCurrentGrowth = 0
        petInteract = FALSE

    GAME VARIABLES
        authChoice = 0
        interactPetChoice = 0
        userChoice = 0
        gameSystemRunning = TRUE
        logIn = FALSE
        optionPicked = FALSE

    WHILE gameSystemRunning = TRUE DO
        EXECUTE PROGRAM
    END WHILE

    DISPLAY "THANK YOU FOR PLAYING"

END GAME LOOP
```

Access Menu

```
BEGIN ACCESS MENU

    WHILE logIn = FALSE DO

        DISPLAY "Signup [1], Login [2], Exit [3]"
        READ authChoice

        IF authChoice is not an integer THEN
            DISPLAY "NOT A NUMBER"
        END IF

        IF authChoice = 1 THEN
            EXECUTE SIGNUP
        ELSE IF authChoice = 2 THEN
```

```

        EXECUTE LOGIN
    ELSE IF authChoice = 3 THEN
        EXECUTE EXIT
    ELSE
        DISPLAY "INVALID CHOICE"
    END IF

END WHILE

END ACCESS MENU

```

User Signup

```

BEGIN USER SIGNUP

    IF userCount < users.length THEN

        DISPLAY "SIGN UP"
        READ newUser
        SET duplicateUser = FALSE

        FOR i FROM 0 TO users.length - 1 DO
            IF users[i].username = newUser THEN
                SET duplicateUser = TRUE
            END IF
            SET i = i + 1
        END FOR

        IF duplicateUser = TRUE THEN
            DISPLAY "USERNAME ALREADY EXISTS"
            SET duplicateUser = FALSE
            CONTINUE
        ELSE
            READ newPassword
            REGISTER newUser with newPassword
            INCREMENT userCount
            DISPLAY "ACCOUNT CREATED"
        END IF

    ELSE
        DISPLAY "MAXIMUM AMOUNT OF USERS"
    END IF

END USER SIGNUP

```

User Login

```

BEGIN USER LOGIN

    IF users[0] = NULL THEN
        DISPLAY "NO USER FOR LOGIN"
        DISPLAY "CREATE AN ACCOUNT FIRSTLY!"
        CONTINUE
    END IF

    DISPLAY "LOG IN"
    READ inputUser
    READ inputPassword

    SET found = FALSE

```

```

FOR i FROM 0 TO users.length - 1 DO
    IF users[i] ≠ NULL AND users[i] = inputUser AND passwords[i] = inputPassword THEN
        SET found = TRUE
        SET currentUser = i
        SET logIn = TRUE
        DISPLAY "LOGIN SUCCESSFULLY"
        DISPLAY "Welcome to PetSim RPG, " + users[currentUser]
        BREAK
    END IF
    SET i = i + 1
END FOR

IF found = FALSE THEN
    DISPLAY "INVALID CREDENTIALS"
END IF

END USER LOGIN

```

User Exit

```

BEGIN USER EXIT

    SET gameSystemRunning = FALSE
    SET logIn = FALSE
    SET optionPicked = FALSE
    SET petInteract = FALSE

    DISPLAY "EXIT GAME"
    DISPLAY "EXITING GAME..."

    BREAK

END USER EXIT

```

PetSim RPG Menu

```

BEGIN PETSIM RPG MENU

    WHILE logIn = TRUE AND optionPicked = FALSE DO

        DISPLAY "-----***** PetSim RPG *****-----"
        DISPLAY "1. | ADOPT A PET"
        DISPLAY "2. | VIEW PET"
        DISPLAY "3. | INTERACT WITH THE PET"
        DISPLAY "4. | RELEASE THE PET"
        DISPLAY "5. | LOGOUT"
        DISPLAY "=====*****===== "
        DISPLAY ">>YOUR CHOICE: "

        IF input is not an integer THEN
            DISPLAY "NOT A NUMBER"
            DISPLAY "Invalid input! Please enter a number."
            CONTINUE
        END IF

        READ userChoice

        IF userChoice = 1 THEN
            EXECUTE ADOPT PET

```

```

ELSE IF userChoice = 2 THEN
    EXECUTE VIEW PET
ELSE IF userChoice = 3 THEN
    EXECUTE INTERACT PET
ELSE IF userChoice = 4 THEN
    EXECUTE RELEASE PET
ELSE IF userChoice = 5 THEN
    EXECUTE LOGOUT
ELSE IF userChoice < 0 AND userChoice > 5
    DISPLAY "INVALID NUMBER CHOICE"
END IF

optionPicked = TRUE

END WHILE

END PETSIM RPG MENU

```

Adopt Pet

```

BEGIN ADOPT PET

WHILE optionPicked = TRUE AND userChoice = 1 DO

    IF hasPet[currentUser] = FALSE THEN

        DISPLAY "-----"
        DISPLAY "|                                     |"
        DISPLAY "|             !!!ADOPT YOUR DESIRED PET AND NAME IT!!!             |"
        DISPLAY "|                                     |"
        DISPLAY "-----"

        DISPLAY "CHOOSE YOUR PET"
        DISPLAY "1. | petType[0]"
        DISPLAY "2. | petType[1]"
        DISPLAY "3. | petType[2]"
        DISPLAY "4. | petType[3]"
        DISPLAY "5. | petType[4]"
        DISPLAY "6. | petType[5]"
        DISPLAY "7. | Exit"
        DISPLAY ">>Your Choice: "

        IF input is not an integer THEN
            DISPLAY "NOT A NUMBER"
            DISPLAY "Invalid input! Please enter a number."
            CONTINUE
        END IF

        READ userPetChoice

        SWITCH userPetChoice
            CASE 1: userPet[currentUser] = petType[0]
            CASE 2: userPet[currentUser] = petType[1]
            CASE 3: userPet[currentUser] = petType[2]
            CASE 4: userPet[currentUser] = petType[3]
            CASE 5: userPet[currentUser] = petType[4]
            CASE 6: userPet[currentUser] = petType[5]
            CASE 7: optionPicked = FALSE; CONTINUE
        DEFAULT:
            DISPLAY "PET CHOICE NOT FOUND"
            DISPLAY "There is no such pet in the choices, Choose Again!"
            CONTINUE
    END IF
END WHILE

```

```

END SWITCH

DISPLAY "ENTER PET NAME"
READ petName[currentUser]
SET hasPet[currentUser] = TRUE

SET petExpCap[currentUser] = 100
SET petExp[currentUser] = 0
SET petLevel[currentUser] = 1
SET petCurrentMood = 1
SET petMood[currentUser] = petDiffMoods[petCurrentMood]
SET petEnergy[currentUser] = 10
SET petEnergyCap[currentUser] = 10
SET petCurrentGrowth = 0
SET petGrowthStatus[currentUser] = petDiffGrowth[petCurrentGrowth]

DISPLAY "YOUR OWN PET"
DISPLAY "Pet Owner: " + users[currentUser]
DISPLAY "Pet Name: " + petName[currentUser]
DISPLAY "Pet Type: " + userPet[currentUser]
DISPLAY "[Take Good Care of your Pet!]"
DISPLAY "Main Goal: Reach LVL 10"

ELSE
    DISPLAY "ALREADY HAVE A PET"
    DISPLAY "You already have a pet -> " + userPet[currentUser] + ": " +
petName[currentUser]
END IF

SET optionPicked = FALSE

END WHILE

END ADOPT PET

```

View Pet

```

BEGIN VIEW PET

WHILE optionPicked = TRUE AND userChoice = 2 DO

    IF hasPet[currentUser] = TRUE THEN
        DISPLAY "PET INFORMATION"
        DISPLAY "Pet Name      : " + petName[currentUser]
        DISPLAY "Pet Type      : " + userPet[currentUser]
        DISPLAY "Pet Level     : " + petLevel[currentUser]
        DISPLAY "Pet Exp       : " + petExp[currentUser] + "/" + petExpCap[currentUser]
        DISPLAY "Growth Status : " + petGrowthStatus[currentUser]
        DISPLAY "Pet Energy    : " + petEnergy[currentUser] + "/" + petEnergyCap[currentUser]
        DISPLAY "Current Mood  : " + petMood[currentUser]
    ELSE
        DISPLAY "PET CANNOT BE FOUND"
        DISPLAY "You don't have a Pet, try and ADOPT one!"
    END IF

    SET optionPicked = FALSE

END WHILE

END VIEW PET

```

Interact Pet

```
BEGIN INTERACTION MENU
```

```
IF hasPet[currentUser] = TRUE THEN
    DISPLAY "INTERACTIONS"
    DISPLAY "1. TRAIN    - Gain EXP (Energy -2)"
    DISPLAY "2. PLAY     - Gain EXP & Mood +1 (Energy -1)"
    DISPLAY "3. FEED     - Restore Energy & Mood"
    DISPLAY "4. REST     - Restore Big Energy"
    DISPLAY "5. GO BACK"
    DISPLAY ">>Enter your choice:"

    IF input is not an integer THEN
        DISPLAY "NOT A NUMBER"
        DISPLAY "Invalid input! Please enter a number."
        CONTINUE
    END IF

    READ interactPetChoice
    SET petInteract = TRUE

ELSE
    DISPLAY "PET CANNOT BE FOUND"
    DISPLAY "You currently have no pet to play with, Try and get one!"
    SET optionPicked = FALSE
    BREAK
END IF
```

```
END INTERACTION MENU
```

Release Pet

```
BEGIN RELEASE PET
```

```
IF optionPicked = TRUE AND userChoice = 4 THEN

    IF hasPet[currentUser] = TRUE THEN
        DISPLAY "RELEASE PET"
        DISPLAY "You will be releasing your pet " + petName[currentUser]
        DISPLAY "!!!WARNING: This will delete your current pet!!!"
        DISPLAY "Enter 'Y' to proceed"
        DISPLAY "Enter 'X' to cancel"
        DISPLAY ">>ENTER CHOICE: "

        READ releaseInput

        IF releaseInput = "Y" THEN
            SET hasPet[currentUser] = FALSE
            DISPLAY "RELEASED PET"
            DISPLAY users[currentUser] + " released " + petName[currentUser] + " into the
wilderness."
            DISPLAY petName[currentUser] + " will forever remember the one who took great care
of him/her."

        ELSE IF releaseInput = "X" THEN
            DISPLAY "CANCELLING RELEASE"
            DISPLAY "GOING BACK TO GAME MENU"

        ELSE
            DISPLAY "INVALID INPUT"
            DISPLAY "Invalid Input, Try Again!"
            CONTINUE
        END IF

    END IF
```



```

ELSE
    DISPLAY "PET CANNOT BE FOUND"
    DISPLAY "You don't have a pet to release, adopt one first."
END IF

SET optionPicked = FALSE

END IF

END RELEASE PET

```

Logout

```

BEGIN LOGOUT

IF optionPicked = TRUE AND userChoice = 5 THEN
    DISPLAY "ACCOUNT LOGOUT"
    DISPLAY users[currentUser] + " LOGGED OUT OF PETSIM RPG..."
    SET optionPicked = FALSE
    SET logIn = FALSE
END IF

END LOGOUT

```

Train Pet

```

BEGIN TRAIN PET

IF petInteract = TRUE AND interactPetChoice = 1 THEN
    DISPLAY "PET TRAINING"

    IF petEnergy[currentUser] > 1 THEN
        DISPLAY users[currentUser] + " trained " + userPet[currentUser] + " " +
petName[currentUser]
        SET expGain = RANDOM number between 0 and 49
        ADD expGain to petExp[currentUser]
        SUBTRACT 2 from petEnergy[currentUser]
        DISPLAY petName[currentUser] + " gained " + expGain + " exp"

    ELSE
        DISPLAY petName[currentUser] + " is too tired to do training"
        DISPLAY petName[currentUser] + ", its mood has lowered"
        DECREMENT petCurrentMood by 1

        IF petCurrentMood < 0 THEN
            SET petCurrentMood = 0
        END IF

        IF petCurrentMood > petDiffMoods.length - 1 THEN
            SET petCurrentMood = petDiffMoods.length - 1
        END IF

        SET petMood[currentUser] = petDiffMoods[petCurrentMood]
    END IF

    DISPLAY "END OF TRAINING"

END IF

```

```
END TRAIN PET
```

Play Pet

```
BEGIN PLAY PET
```

```
IF petInteract = TRUE AND interactPetChoice = 2 THEN
    DISPLAY "PLAYING WITH PET"

    IF petEnergy[currentUser] > 0 THEN
        DISPLAY users[currentUser] + " played with " + userPet[currentUser] + " " +
petName[currentUser]
        SET expGain = RANDOM number between 0 and 19
        ADD expGain to petExp[currentUser]
        SUBTRACT 1 from petEnergy[currentUser]
        INCREMENT petCurrentMood by 1
        DISPLAY petName[currentUser] + " gained " + expGain + " exp"

    ELSE
        DISPLAY petName[currentUser] + " is too tired to play with " + users[currentUser]
        DISPLAY petName[currentUser] + "'s mood has lowered"
        DECREMENT petCurrentMood by 1
    END IF

    IF petCurrentMood < 0 THEN
        SET petCurrentMood = 0
    END IF

    IF petCurrentMood > petDiffMoods.length - 1 THEN
        SET petCurrentMood = petDiffMoods.length - 1
    END IF

    SET petMood[currentUser] = petDiffMoods[petCurrentMood]

    DISPLAY "END OF PLAY SESSION"

END IF

END PLAY PET
```

Feed Pet

```
BEGIN FEED PET
```

```
IF petInteract = TRUE AND interactPetChoice = 3 THEN
    DISPLAY "FEEDING PET"
    DISPLAY users[currentUser] + " fed " + userPet[currentUser] + " " + petName[currentUser]

    SET energyGain = RANDOM number between 0 and 1
                    + ((petEnergyCap[currentUser] - 10) / 3)
                    + 1
    ADD energyGain to petEnergy[currentUser]

    IF petEnergy[currentUser] > petEnergyCap[currentUser] THEN
        SET petEnergy[currentUser] = petEnergyCap[currentUser]
    END IF

    IF petCurrentMood < petDiffMoods.length - 1 THEN
        INCREMENT petCurrentMood by 1
        SET petMood[currentUser] = petDiffMoods[petCurrentMood]
    END IF
END IF
```

```

ELSE
    SET petMood[currentUser] = petDiffMoods[petCurrentMood]
END IF

DISPLAY petName[currentUser] + " eats the food happily!"
DISPLAY petName[currentUser] + " gained " + energyGain + " energy"
DISPLAY "END OF FEEDING"

END IF

END FEED PET

```

Rest Pet

```

BEGIN REST PET

IF petInteract = TRUE AND interactPetChoice = 4 THEN
    DISPLAY "RESTING PET"
    DISPLAY petName[currentUser] + " sleeps gently on its bed"

    SET energyGain = RANDOM number between 0 and 4
                    + ((petEnergyCap[currentUser] - 10) / 2)
                    + 1
    ADD energyGain to petEnergy[currentUser]

    IF petEnergy[currentUser] > petEnergyCap[currentUser] THEN
        SET petEnergy[currentUser] = petEnergyCap[currentUser]
    END IF

    DISPLAY petName[currentUser] + " gained " + energyGain + " energy"
    DISPLAY "END OF REST SESSION"
END IF

END REST PET

```

Go Back

```

BEGIN GO BACK

IF petInteract = TRUE AND interactPetChoice = 5 THEN
    SET optionPicked = FALSE
    SET petInteract = FALSE
    BREAK from interaction loop
END IF

END GO BACK

```

After Action

BEGIN AFTER ACTION

```
IF hasPet[currentUser] = TRUE THEN

  IF petExp[currentUser] >= petExpCap[currentUser] THEN
    SUBTRACT petExpCap[currentUser] from petExp[currentUser]
    INCREMENT petLevel[currentUser] by 1
    SET petExpCap[currentUser] = 100 + (petLevel[currentUser] - 1) * 25
    INCREMENT petEnergyCap[currentUser] by 1
    SET petEnergy[currentUser] = 10 + (petEnergyCap[currentUser] - 10)
    DISPLAY "LEVEL UP"
    DISPLAY "Congrats! " + petName[currentUser] + " leveled up!"
  END IF

  IF (petLevel[currentUser] = 3 AND petCurrentGrowth < 1)
    OR (petLevel[currentUser] = 6 AND petCurrentGrowth < 2)
    OR (petLevel[currentUser] = 9 AND petCurrentGrowth < 3)
    AND (petCurrentGrowth < petDiffGrowth.length - 1) THEN
    INCREMENT petCurrentGrowth by 1
    SET petGrowthStatus[currentUser] = petDiffGrowth[petCurrentGrowth]
    DISPLAY "PET GROWTH"
    DISPLAY petName[currentUser] + "'s growth status is now " + petGrowthStatus[currentUser]
  ELSE IF petLevel[currentUser] >= 10 THEN
    SET finishedGoal[currentUser] = TRUE
    SET optionPicked = FALSE
    SET petInteract = FALSE
    SET logIn = FALSE
    DISPLAY "GOAL REACHED"
    DISPLAY users[currentUser] + " attained 'Master Trainer' with " + petName[currentUser] +
    " at LVL " + petLevel[currentUser]
  END IF

  IF finishedGoal[currentUser] = FALSE THEN
    DISPLAY "ACTION RESULT"
    DISPLAY "Level           : " + petLevel[currentUser]
    DISPLAY "Growth Status    : " + petGrowthStatus[currentUser]
    DISPLAY "Exp              : " + petExp[currentUser] + "/" + petExpCap[currentUser]
    DISPLAY "Energy           : " + petEnergy[currentUser] + "/" + petEnergyCap[currentUser]
    DISPLAY "Mood             : " + petMood[currentUser]
  END IF

END IF

SET petInteract = FALSE
END AFTER ACTION
```

E. Code

BETTER READABILITY

```
package GameSysCRUD;

//Programmed by: Mike Chester M. Dumail

import java.util.Scanner;

public class LogInTest {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        /*
         * Usage of "System.out.println("\n");" -> is to give a breathing space for every
         * action or display attempt. The reason is for readability and documenting the
         * console output.
         */

        /*
         * Using the idea of parallel arrays, so that for each user they have their own
         * respective place or index in the arrays. Simply said, each user is assigned
         * an index for all the related game mechanics, acting as a save file.
         */

        int capacity = 3; //Change to set how much users the program can handle

        //USER RELATED VARIABLES
        String [] users = new String[capacity];
        String [] passwords = new String[capacity];
        boolean [] hasPet = new boolean[capacity];
        boolean [] finishedGoal = new boolean[capacity];
        String [] userPet = new String[capacity];

        boolean duplicateUser = false;
        int currentUser = 0;
        int userCount = 0;
        int userID = 0;

        //PET RELATED VARIABLES
        String [] petType = {"Cat", "Dog", "Duck", "Pig", "Rabbit", "Horse"};
        String [] petDiffMoods = {"Sad", "Neutral", "Happy", "Joyful"};
        String [] petDiffGrowth= {"Baby", "Child", "Teen", "Adult"};

        String [] petGrowthStatus = new String[capacity];
        String [] petMood = new String[capacity];
        String [] petName = new String[capacity];

        int [] petExpCap = new int[capacity];
        int [] petExp = new int[capacity];
        int [] petLevel = new int[capacity];
        int [] petEnergy = new int[capacity];
        int [] petEnergyCap = new int[capacity];

        int petCurrentMood = 0;
        int petCurrentGrowth = 0;

        boolean petInteract = false;

        //GAME PROGRAM RELATED VARIABLES
        int authChoice = 0;
        int interactPetChoice = 0;
        int userChoice = 0;

        boolean gameSystemRunning = true;
        boolean logIn = false;
        boolean optionPicked = false;

        //MAIN MENU -> THE HANDLER OF IT ALL
        while (gameSystemRunning) {

            //LOGIN SYSTEM
            while(!logIn){

                System.out.println("===== WELCOME TO PETSIM RPG =====");
                System.out.print("1. | SIGNUP" + "\n" +
                                "2. | LOGIN" + "\n" +
                                "3. | EXIT" + "\n" +
                                "===== " + "\n" +
                                ">>>YOUR CHOICE: ");

                if(!sc.hasNextInt()) {
                    System.out.println("\n");
                    System.out.println("===== NOT A NUMBER =====");
                    System.out.println("\n    Invalid input! Please enter a number. \n");
                    System.out.println("=====");
                    System.out.println("\n");
                    sc.next();
                    continue;
                }

                authChoice = sc.nextInt();
                sc.nextLine();
                System.out.println("\n");
            }
        }
    }
}
```

```

if(authChoice == 1){
    if(userCount < users.length){

        System.out.println("===== SIGN UP =====");
        System.out.print("Create Username: ");
        String newUser = sc.nextLine();

        for(int i = 0; i < users.length; i++){
            if(newUser.equals(users[i])) duplicateUser = true;
        }

        if(duplicateUser){
            System.out.println("\n");
            System.out.println("----- USERNAME ALREADY EXISTS -----");
            System.out.println("\n      " + newUser + " is already taken, input another one \n");
            System.out.println("-----");
            System.out.println("\n");
            duplicateUser = false;
            continue;
        }

        System.out.print("Create Your Password: ");
        String newPassword = sc.nextLine();

        System.out.println("=====");
        System.out.println("\n");

        users[userCount] = newUser;
        passwords[userCount] = newPassword;

        userCount++;
        userID = userCount;

        System.out.println("***** ACCOUNT CREATED *****");
        System.out.println("\n" + "\t#User_" + userID + ": " + newUser + "\n");
        System.out.println("*****");
        System.out.println("\n");
    }
    else{
        System.out.println("===== MAX USER =====");
        System.out.println("\n      !!MAXIMUM AMOUNT OF USERS!! \n");
        System.out.println("=====");
        System.out.println("\n");
    }
}
else if(authChoice == 2){

    //EMPTY ARRAY CHECK -> [0] ARRAY CHECK BECAUSE OF CONTINUITY
    if(users[0] == null){
        System.out.println("===== NO USER FOR LOGIN =====");
        System.out.println("\n      CREATE AN ACCOUNT FIRSTLY! \n");
        System.out.println("=====");

        System.out.println("\n");
        continue;
    }

    System.out.println("===== LOG IN =====");
    System.out.print("Enter Username: ");
    String inputUser = sc.nextLine();

    System.out.print("Enter Password: ");
    String inputPassword = sc.nextLine();
    System.out.println("=====");

    System.out.println("\n");

    //CHECK FOR USER AND PASSWORD VALIDITY
    boolean found = false;
    for(int i = 0; i < users.length; i++){
        if(users[i] != null && users[i].equals(inputUser) && passwords[i].equals(inputPassword)){
            found = true;
            currentUser = i;
            login = true;

            System.out.println("===== LOGIN SUCCESSFULLY =====");
            System.out.println("\n" + "      Welcome to PetSim RPG, " + users[currentUser] + "\n");
            System.out.println("=====");

            System.out.println("\n");
            break;
        }
    }
    if(!found){
        System.out.println("===== INVALID =====");
        System.out.println("\n      INVALID CREDENTIALS! \n");
        System.out.println("=====");

        System.out.println("\n");
    }
}
else if(authChoice == 3){
    gameSystemRunning = false;
    login = false;
    optionPicked = false;
    petInteract = false;

    System.out.println("===== EXIT GAME =====");
    System.out.println("\n      EXITING GAME... \n");
    System.out.println("=====");
}

```

```

        System.out.println("\n");
        break;
    }
    else{
        System.out.println("===== INVALID NUMBER =====");
        System.out.println("\n          INVALID NUMBER, TRY AGAIN          \n");
        System.out.println("=====");

        System.out.println("\n");
    }
}

//login menu end

/*
 * This section below, you have successfully log-in to your account.
 */

//OPTIONS MENU
while(logIn && !optionPicked){

    System.out.println("-----***** PetSim RPG *****-----");
    System.out.print("1. | ADOPT A PET" + "\n" +
        "2. | VIEW PET" + "\n" +
        "3. | INTERACT WITH THE PET" + "\n" +
        "4. | RELEASE THE PET" + "\n" +
        "5. | LOGOUT" + "\n");
    System.out.println("=====");
    System.out.print(">>>YOUR CHOICE: ");

    if(!sc.hasNextInt()) {
        System.out.println("\n");
        System.out.println("===== NOT A NUMBER =====");
        System.out.println("\n    Invalid input! Please enter a number. \n");
        System.out.println("=====");
        System.out.println("\n");
        sc.next();
        continue;
    }

    userChoice = sc.nextInt();
    sc.nextLine();

    System.out.println("\n");

    optionPicked = true;
}

//game menu end

//ADOPT PET OPTION
while (optionPicked && userChoice == 1) {
    if (hasPet[currentUser] == false) {

        //GAME MESSAGE
        System.out.println("""
            |-----|
            |  !!!ADOPT YOUR DESIRED PET AND NAME IT!!!  |
            |-----|
            """);

        System.out.println("\n");

        System.out.println("----- CHOOSE YOUR PET -----");
        System.out.print("1. | " + petType[0] + "\n" +
            "2. | " + petType[1] + "\n" +
            "3. | " + petType[2] + "\n" +
            "4. | " + petType[3] + "\n" +
            "5. | " + petType[4] + "\n" +
            "6. | " + petType[5] + "\n" +
            "7. | " + "Exit" + "\n");

        System.out.println("-----");
        System.out.print(">>>Your Choice: ");

        if(!sc.hasNextInt()) {
            System.out.println("\n");
            System.out.println("===== NOT A NUMBER =====");
            System.out.println("\n    Invalid input! Please enter a number. \n");
            System.out.println("=====");
            System.out.println("\n");
            sc.next();
            continue;
        }

        int userPetChoice = sc.nextInt();
        sc.nextLine();

        System.out.println("\n");

        //CHOOSING PET
        switch (userPetChoice) {
            case 1 -> userPet[currentUser] = petType[0];
            case 2 -> userPet[currentUser] = petType[1];
            case 3 -> userPet[currentUser] = petType[2];
            case 4 -> userPet[currentUser] = petType[3];
            case 5 -> userPet[currentUser] = petType[4];
            case 6 -> userPet[currentUser] = petType[5];
            case 7 -> { optionPicked = false; continue; }
            default -> {

```

```

        System.out.println("----- PET CHOICE NOT FOUND -----");
        System.out.println("\nThere is no such pet in the choices, Choose Again!" + "\n");
        System.out.println("-----");
        System.out.println("\n");
        continue;
    }
}

System.out.println("====^^^ PET NAME ^^^====");
System.out.print("Enter Pet Name: ");
petName[currentUser] = sc.nextLine();
hasPet[currentUser] = true;
System.out.println("=====");

System.out.println("\n");

//INITIAL PET STATS
petExpCap[currentUser] = 100;
petExp[currentUser] = 0;
petLevel[currentUser] = 1;
petCurrentMood = 1;
petMood[currentUser] = petDiffMoods[petCurrentMood];
petEnergy[currentUser] = 10;
petEnergyCap[currentUser] = 10;
petCurrentGrowth = 0;
petGrowthStatus[currentUser] = petDiffGrowth[petCurrentGrowth];

System.out.println("***** YOUR OWN PET *****");
System.out.println("Pet Name: " + petName[currentUser]);
System.out.println("Pet Type: " + userPet[currentUser]);
System.out.println("\n [Take Good Care of your Pet!] \n");
System.out.println("Main Goal: Reach LVL 10 ");
System.out.println("*****");

System.out.println("\n");

}
else{
    System.out.println("----- ALREADY HAVE A PET -----");
    System.out.println("\n You already have a pet -> " + userPet[currentUser] + ": " + petName[currentUser]);
    System.out.println("-----");
    System.out.println("\n");
}

optionPicked = false;
} //adopt pet end

//VIEW PET OPTION
while(optionPicked && userChoice == 2){
    if(hasPet[currentUser] == true){
        System.out.println("----- PET INFORMATION -----");
        System.out.println("Pet Owner : " + users[currentUser]);
        System.out.println("Pet Name : " + petName[currentUser]);
        System.out.println("Pet Type : " + userPet[currentUser]);
        System.out.println("Pet Level : " + petLevel[currentUser]);
        System.out.println("Pet Exp : " + petExp[currentUser] + "/" + petExpCap[currentUser]);
        System.out.println("Growth Status : " + petGrowthStatus[currentUser]);
        System.out.println("Pet Energy : " + "[" + "]" + ".repeat(petEnergy[currentUser]) + " ".repeat(petEnergyCap[currentUser] -
petEnergy[currentUser]) + "]" + " -> " + petEnergy[currentUser] + "/" + petEnergyCap[currentUser]);
        System.out.println("Current Mood : " + petMood[currentUser]);
        System.out.println("-----");
        System.out.println("\n");
    }
    else{
        System.out.println("----- PET CANNOT BE FOUND -----");
        System.out.println("\n It seems that you don't have a Pet, try and ADOPT one! \n");
        System.out.println("-----");
        System.out.println("\n");
    }
}

optionPicked = false;
} //view pet end

//INTERACT WITH PET OPTION
while (optionPicked && userChoice == 3) {

    //STOP INTERACTING AFTER REACHING MAIN GOAL
    if(finishedGoal[currentUser]){
        System.out.println("===== MASTER TRAINER =====");
        System.out.println("\n You have achieved the pinnacle of pet training \n");
        System.out.println("=====");

        System.out.println("\n");

        optionPicked = false;
        break;
    }

    //INTERACTION WITH PET MENU
    if(hasPet[currentUser] == true){
        System.out.println("----- INTERACTIONS -----");
        System.out.print("1. | TRAIN - Gain EXP (Energy -2)" + "\n" +
"2. | PLAY - Gain EXP & Mood +1 (Energy -1)" + "\n" +
"3. | FEED - Restore Energy & Mood" + "\n" +
"4. | REST - Restore Big Energy" + "\n" +
"5. | GO BACK" + "\n");
        System.out.println("-----");
    }
}

```



```

System.out.print(">>Enter your choice: ");

if(!sc.hasNextInt()) {
    System.out.println("\n");
    System.out.println("===== NOT A NUMBER =====");
    System.out.println("\n    Invalid input! Please enter a number. \n");
    System.out.println("=====");
    System.out.println("\n");
    sc.next();
    continue;
}

interactPetChoice = sc.nextInt();
sc.nextLine();

System.out.println("\n");

petInteract = true;
}
else{
    System.out.println("----- PET CANNOT BE FOUND -----");
    System.out.println("\n    You currently have no pet to play with, Try and get one! \n");
    System.out.println("-----");

    System.out.println("\n");
    optionPicked = false;
    break;
}
}
}

//TRAIN OPTION
if(petInteract && interactPetChoice == 1){
    System.out.println("----- PET TRAINING -----");

    if(petEnergy[currentUser] > 1){
        System.out.println("\n    " + users[currentUser] + " trained " + userPet[currentUser] +
            "\n    " + petName[currentUser] + "");

        int expGain = (int)(Math.random()*50);
        petExp[currentUser] += expGain;
        petEnergy[currentUser] -= 2;

        System.out.println("\n    " + petName[currentUser] + " gained " + expGain + " exp " + "\n");
    }
    else{
        System.out.println("\n    " + petName[currentUser] + " is too tired to do training " + "\n");
        System.out.println("\n    " + petName[currentUser] + ", its mood has lowered :( " + "\n");
        petCurrentMood -= 1;

        if (petCurrentMood < 0) petCurrentMood = 0;
        if (petCurrentMood > petDiffMoods.length - 1) petCurrentMood = petDiffMoods.length - 1;

        petMood[currentUser] = petDiffMoods[petCurrentMood];
    }

    System.out.println("-----");

    System.out.println("\n");
}
}

//PLAY OPTION
else if(petInteract && interactPetChoice == 2){
    System.out.println("----- PLAYING WITH PET -----");

    if(petEnergy[currentUser] > 0){
        System.out.println("\n    " + users[currentUser] + " played with " + userPet[currentUser] + " " + petName[currentUser] + "");
        int expGain = (int)(Math.random()*20);
        petExp[currentUser] += expGain;
        petEnergy[currentUser] -= 1;
        petCurrentMood += 1;

        System.out.println("\n    " + petName[currentUser] + " gained " + expGain + " exp " + "\n");
    }
    else{
        System.out.println("\n    " + petName[currentUser] + " is too tired to play with " + users[currentUser]);
        System.out.println("\n    " + petName[currentUser] + "'s mood has lowered :( " + "\n");
        petCurrentMood -= 1;
    }

    if (petCurrentMood < 0) petCurrentMood = 0;
    if (petCurrentMood > petDiffMoods.length - 1) petCurrentMood = petDiffMoods.length - 1;

    petMood[currentUser] = petDiffMoods[petCurrentMood];

    System.out.println("-----");

    System.out.println("\n");
}

//FEED OPTION
else if(petInteract && interactPetChoice == 3){
    System.out.println("----- FEEDING PET -----");

    System.out.println("\n    " + users[currentUser] + " fed " + userPet[currentUser] + " " + petName[currentUser] + "");
    int energyGain = (int)(Math.random()*2) + ((petEnergyCap[currentUser] - 10)/3) + 1;
    petEnergy[currentUser] += energyGain;

    if(petEnergy[currentUser] > petEnergyCap[currentUser]){
        petEnergy[currentUser] = petEnergyCap[currentUser];
    }
}
}

```

```

        if(petCurrentMood < petDiffMoods.length - 1){
            petCurrentMood += 1;
            petMood[currentUser] = petDiffMoods[petCurrentMood];
        }
        else{
            petMood[currentUser] = petDiffMoods[petCurrentMood];
        }

        System.out.println("*****");
        System.out.println("\n      " + petName[currentUser] + " eats the food happily!");
        System.out.println("\n      " + petName[currentUser] + " gained " + energyGain + " energy" + "\n");
        System.out.println("*****");

        System.out.println();

        System.out.println("-----");

        System.out.println("\n");
    } //feed pet end

    //REST OPTION
    else if(petInteract && interactPetChoice == 4){
        System.out.println("----- RESTING PET -----");

        System.out.println("\n      " + petName[currentUser] + "\" " + " sleeps gently on its bed" + "\n");
        int energyGain = (int)(Math.random()*5) + ((petEnergyCap[currentUser] - 10)/2) + 1;
        petEnergy[currentUser] += energyGain;

        if(petEnergy[currentUser] > petEnergyCap[currentUser]){
            petEnergy[currentUser] = petEnergyCap[currentUser];
        }

        System.out.println("*****");
        System.out.println("\n      " + petName[currentUser] + " gained " + energyGain + " energy" + "\n");
        System.out.println("*****");

        System.out.println();

        System.out.println("-----");

        System.out.println("\n");
    } //rest pet end

    //GO BACK OPTION
    else if(petInteract && interactPetChoice == 5){
        optionPicked = false;
        petInteract = false;
        break;
    } //go back end

    //INVALID NUMBER CHOICE INPUT
    else{
        System.out.println("===== INVALID CHOICE INPUT =====");
        System.out.println("\n      INVALID INPUT, PLEASE ENTER A VALID INPUT! \n");
        System.out.println("=====");
        System.out.println("\n");
        continue;
    }
}

//AFTER INTERACTIONS
if(hasPet[currentUser] == true){

    //LEVEL UP CHECKER
    if(petExp[currentUser] >= petExpCap[currentUser]){

        petExp[currentUser] -= petExpCap[currentUser];
        petLevel[currentUser]++;

        petExpCap[currentUser] = 100 + (petLevel[currentUser] - 1) * 25;

        petEnergyCap[currentUser]++;
        petEnergy[currentUser] = 10 + (petEnergyCap[currentUser] - 10);

        System.out.println("***** LEVEL UP *****");
        System.out.println("\n      Congrats! " + petName[currentUser] + " leveled up! \n");
        System.out.println("*****");

        System.out.println("\n");
    }

    //GOAL CHECKER
    if((petLevel[currentUser] == 3 && petCurrentGrowth < 1) ||
        (petLevel[currentUser] == 6 && petCurrentGrowth < 2) ||
        (petLevel[currentUser] == 9 && petCurrentGrowth < 3) &&
        (petCurrentGrowth < petDiffGrowth.length - 1)) {

        petCurrentGrowth += 1;
        petGrowthStatus[currentUser] = petDiffGrowth[petCurrentGrowth];

        System.out.println("***** PET GROWTH *****");
        System.out.println("\n      " + petName[currentUser] + "'s growth status is now " + petGrowthStatus[currentUser] + "\n");
        System.out.println("*****");

        System.out.println("\n");
    }
}
else if(petLevel[currentUser] >= 10){

```

```

        finishedGoal[currentUser] = true;
        optionPicked = false;
        petInteract = false;
        logIn = false;

        System.out.println("===== GOAL REACHED =====");
        System.out.println("\n" + users[currentUser] + " has attained \"Master Trainer\" for having " + "\n"
            + petName[currentUser] + " reached LVL " + petLevel[currentUser] + "\n");
        System.out.println("=====");

        System.out.println("\n");
    }

    //DISPLAY
    if(!finishedGoal[currentUser]){
        System.out.println("===== ACTION RESULT =====" + "\n" +
            "Level      : " + petLevel[currentUser] + "\n" +
            "Growth Status : " + petGrowthStatus[currentUser] + "\n" +
            "Exp          : " + petExp[currentUser] + "/" + petExpCap[currentUser] + "\n" +
            "Energy       : " + "[" + "]" + ".repeat(petEnergy[currentUser]) + " ".repeat(petEnergyCap[currentUser] -
petEnergy[currentUser]) + "]" + " -> " + petEnergy[currentUser] + "/" + petEnergyCap[currentUser] + "\n" +
            "Mood         : " + petMood[currentUser]);
        System.out.println("=====");

        System.out.println("\n");
    } //after interactions end

    }

    petInteract = false;
} //interact pet end

//RELEASE PET OPTION
if(optionPicked && userChoice == 4){
    if(hasPet[currentUser] == true){
        System.out.println("----- RELEASE PET -----");
        System.out.println("\n      You will be releasing your pet " + petName[currentUser] + "\n");
        System.out.println("!!!WARNING: This will delete your current pet!!!");
        System.out.println("-----");
        System.out.println("Enter \"Y\" to proceed");
        System.out.println("Enter \"X\" to cancel");
        System.out.println("-----");
        System.out.print(">>ENTER CHOICE: ");
        String releaseInput = sc.nextLine();

        System.out.println("\n");

        if(releaseInput.equalsIgnoreCase("Y")){
            hasPet[currentUser] = false;

            System.out.println("----- RELEASED PET -----");
            System.out.println(users[currentUser] + " released " + petName[currentUser] + " into the wilderness." + "\n" +
                petName[currentUser] + " will forever remember the one who took great care of him/her.^^");
            System.out.println("-----");
        }
        else if(releaseInput.equalsIgnoreCase("X")) {
            System.out.println("----- CANCELLING RELEASE -----");
            System.out.println("\n      GOING BACK TO GAME MENU \n");
            System.out.println("-----");
        }
        else{
            System.out.println("===== INVALID INPUT =====");
            System.out.println("\n      Invalid Input, Try Again!      \n");
            System.out.println("=====");
            continue;
        }
    }
    else{
        System.out.println("----- PET CANNOT BE FOUND -----");
        System.out.println("\n      You don't have a pet to release, adopt one first. \n");
        System.out.println("-----");
    }

    System.out.println("\n");

    optionPicked = false;
} //release pet end

//LOGOUT OPTION
if(optionPicked && userChoice == 5){
    System.out.println("----- ACCOUNT LOGOUT -----");
    System.out.println("\n      " + users[currentUser] + " LOGGED OUT OF PETSIM RPG... \n");
    System.out.println("-----");
    System.out.println("\n");

    optionPicked = false;
    logIn = false;
} //logout option end

//OUT OF BOUNDS HANDLING
if(optionPicked && (userChoice < 1 || userChoice > 5)){
    System.out.println("===== INVALID NUMBER CHOICE =====");
    System.out.println("\n      INVALID NUMBER CHOICE, TRY AGAIN!! \n");
    System.out.println("=====");
    System.out.println("\n");
}

```

```

        optionPicked = false;
    }

} //main menu end

//LAST MESSAGE
System.out.println("=====");
System.out.println("      THANKYOU FOR PLAYING PETSIM RPG!      ");
System.out.println("=====");

sc.close();
}
}

```

BETTER ORGANIZATION

```

package GameSysCRUD;

//Programmed by: Mike Chester M. Dumail

import java.util.Scanner;

public class PetSimRPG {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int capacity = 3;

        //USER RELATED VARIABLES
        String [] users = new String[capacity];
        String [] passwords = new String[capacity];

        // 2D ARRAY: CREDENTIALS
        // Row 0: Users, Row 1: Passwords
        String [][] credentials = {
            users,
            passwords
        };

        boolean duplicateUser = false;
        int currentUser = 0;
        int userCount = 0;
        int userID = 0;

        //PET STRING VARIABLES (DEFINITIONS)
        String [] petType = {"Cat", "Dog", "Duck", "Pig", "Rabbit", "Horse"};
        String [] petDiffMoods = {"Sad", "Neutral", "Happy", "Joyful"};
        String [] petDiffGrowth= {"Baby", "Child", "Teen", "Adult"};

        // 2D ARRAY: PET DEFINITIONS
        // Row 0: Types, Row 1: Moods, Row 2: Growth Stages
        String [][] petDefinitions = {
            petType,
            petDiffMoods,
            petDiffGrowth
        };

        //PET STRING VARIABLES (DATA)
        String [] petName = new String[capacity];
        String [] userPet = new String[capacity];
        String [] petGrowthStatus = new String[capacity];
        String [] petMood = new String[capacity];

        // 2D ARRAY: PET STRING DATA
        // Row 0: Name, Row 1: Type(userPet), Row 2: GrowthStatus, Row 3: Mood
        String [][] petStrData = {
            petName,
            userPet,
            petGrowthStatus,
            petMood
        };

        //PET INT VARIABLES
        int petCurrentMood = 0;
        int petCurrentGrowth = 0;

        int [] petLevel = new int[capacity];
        int [] petExp = new int[capacity];
        int [] petExpCap = new int[capacity];
        int [] petEnergy = new int[capacity];
        int [] petEnergyCap = new int[capacity];

        // 2D ARRAY: PET INT DATA
        // [0]Level, [1]Exp, [2]ExpCap, [3]Energy, [4]EnergyCap
        int [][] petIntData = {
            petLevel,
            petExp,
            petExpCap,
            petEnergy,
            petEnergyCap
        };

        //PET BOOL VARIABLES
        boolean [] hasPet = new boolean[capacity];
        boolean [] finishedGoal = new boolean[capacity];
    }
}

```

```

// 2D ARRAY: PET BOOLEAN DATA
// Row 0: hasPet, Row 1: finishedGoal
boolean [][] petBoolData = {
    hasPet,
    finishedGoal
};

//GAME PROGRAM RELATED VARIABLES
int authChoice = 0;
int interactPetChoice = 0;
int userChoice = 0;

boolean gameSystemRunning = true;
boolean login = false;
boolean optionPicked = false;
boolean petInteract = false;

//GAME SYSTEM -> THE HANDLER OF IT ALL
while (gameSystemRunning) {

    //LOGIN SYSTEM
    while(!login){

        System.out.println("===== WELCOME TO PETSIM RPG =====");
        System.out.print("1. | SIGNUP" + "\n" +
            "2. | LOGIN" + "\n" +
            "3. | EXIT" + "\n" +
            "===== " + "\n" +
            ">>>YOUR CHOICE: ");

        if(!sc.hasNextInt()) {
            System.out.println("\n");
            System.out.println("===== NOT A NUMBER =====");
            System.out.println("\n Invalid input! Please enter a number. \n");
            System.out.println("=====");
            System.out.println("\n");
            sc.next();
            continue;
        }

        authChoice = sc.nextInt();
        sc.nextLine();
        System.out.println("\n");

        if(authChoice == 1){
            if(userCount < credentials[0].length){

                System.out.println("===== SIGN UP =====");
                System.out.print("Create Username: ");
                String newUser = sc.nextLine();

                for(int i = 0; i < credentials[0].length; i++){
                    if(newUser.equals(credentials[0][i])) duplicateUser = true;
                }

                if(duplicateUser){
                    System.out.println("\n");
                    System.out.println("----- USERNAME ALREADY EXISTS -----");
                    System.out.println("\n " + newUser + " is already taken, input another one \n");
                    System.out.println("-----");
                    System.out.println("\n");
                    duplicateUser = false;
                    continue;
                }

                System.out.print("Create Your Password: ");
                String newPassword = sc.nextLine();

                System.out.println("=====");
                System.out.println("\n");

                credentials[0][userCount] = newUser;
                credentials[1][userCount] = newPassword;

                userCount++;
                userID = userCount;

                System.out.println("***** ACCOUNT CREATED *****");
                System.out.println("\n" + "\t#User_" + userID + ": " + newUser + "\n");
                System.out.println("*****");
                System.out.println("\n");

            }
            else{
                System.out.println("===== MAX USER =====");
                System.out.println("\n !!!MAXIMUM AMOUNT OF USERS!!! \n");
                System.out.println("=====");
                System.out.println("\n");
            }
        }
        else if(authChoice == 2){

            //EMPTY ARRAY CHECK -> [0] ARRAY CHECK BECAUSE OF CONTINUITY
            if(credentials[0][0] == null){
                System.out.println("===== NO USER FOR LOGIN =====");
                System.out.println("\n CREATE AN ACCOUNT FIRSTLY! \n");
                System.out.println("=====");

                System.out.println("\n");
            }
        }
    }
}

```



```

System.out.println("""
    -----
    |                                     |
    |             !!!ADOPT YOUR DESIRED PET AND NAME IT!!!             |
    |                                     |
    -----
""");
System.out.println("\n");

System.out.println("----- CHOOSE YOUR PET -----");
System.out.print("1. | " + petDefinitions[0][0] + "\n" +
    "2. | " + petDefinitions[0][1] + "\n" +
    "3. | " + petDefinitions[0][2] + "\n" +
    "4. | " + petDefinitions[0][3] + "\n" +
    "5. | " + petDefinitions[0][4] + "\n" +
    "6. | " + petDefinitions[0][5] + "\n" +
    "7. | " + "Exit" + "\n");
System.out.println("-----");
System.out.print(">>Your Choice: ");

if(!sc.hasNextInt()) {
    System.out.println("\n");
    System.out.println("===== NOT A NUMBER =====");
    System.out.println("\n Invalid input! Please enter a number. \n");
    System.out.println("=====");
    System.out.println("\n");
    sc.next();
    continue;
}

int userPetChoice = sc.nextInt();
sc.nextLine();

System.out.println("\n");

//CHOOSING PET
switch (userPetChoice) {
    case 1 -> petStrData[1][currentUser] = petDefinitions[0][0];
    case 2 -> petStrData[1][currentUser] = petDefinitions[0][1];
    case 3 -> petStrData[1][currentUser] = petDefinitions[0][2];
    case 4 -> petStrData[1][currentUser] = petDefinitions[0][3];
    case 5 -> petStrData[1][currentUser] = petDefinitions[0][4];
    case 6 -> petStrData[1][currentUser] = petDefinitions[0][5];
    case 7 -> { optionPicked = false; continue; }
    default -> {
        System.out.println("----- PET CHOICE NOT FOUND -----");
        System.out.println("\nThere is no such pet in the choices, Choose Again!" + "\n");
        System.out.println("-----");
        System.out.println("\n");
        continue;
    }
}

System.out.println("====^^^ PET NAME ^^^====");
System.out.print("Enter Pet Name: ");
petStrData[0][currentUser] = sc.nextLine();
petBoolData[0][currentUser] = true;
System.out.println("=====");

System.out.println("\n");

//INITIAL PET STATS
petIntData[2][currentUser] = 100;
petIntData[1][currentUser] = 0;
petIntData[0][currentUser] = 1;
petCurrentMood = 1;
petStrData[3][currentUser] = petDefinitions[1][petCurrentMood];
petIntData[3][currentUser] = 10;
petIntData[4][currentUser] = 10;
petCurrentGrowth = 0;
petStrData[2][currentUser] = petDefinitions[2][petCurrentGrowth];

System.out.println("***** YOUR OWN PET *****");
System.out.println("Pet Name: " + petStrData[0][currentUser]);
System.out.println("Pet Type: " + petStrData[1][currentUser]);
System.out.println("\n [Take Good Care of your Pet! \n");
System.out.println(" Main Goal: Reach LVL 10 ");
System.out.println("*****");

System.out.println("\n");

}
else{
    System.out.println("----- ALREADY HAVE A PET -----");
    System.out.println("\n You already have a pet -> " + petStrData[1][currentUser] + ": " + petStrData[0][currentUser]);
    System.out.println("-----");
    System.out.println("\n");
}

optionPicked = false;
} //adopt pet end

//VIEW PET OPTION
while(optionPicked && userChoice == 2){
    // CHANGED: hasPet -> petBoolData[0]
    if(petBoolData[0][currentUser] == true){
        System.out.println("----- PET INFORMATION -----");
        System.out.println("Pet Owner : " + credentials[0][currentUser]);
        System.out.println("Pet Name : " + petStrData[0][currentUser]);
    }
}

```

```

        System.out.println("Pet Type      : " + petStrData[1][currentUser]);
        System.out.println("Pet Level   : " + petIntData[0][currentUser]);
        System.out.println("Pet Exp    : " + petIntData[1][currentUser] + "/" + petIntData[2][currentUser]);
        System.out.println("Growth Status : " + petStrData[2][currentUser]);
        int currentEn = petIntData[3][currentUser];
        int capEn = petIntData[4][currentUser];
        System.out.println("Pet Energy    : " + "[" + ".repeat(currentEn) + " ".repeat(capEn - currentEn) + "]" + " -> " + currentEn + "/" +
capEn);

        System.out.println("Current Mood   : " + petStrData[3][currentUser]);
        System.out.println("-----");
        System.out.println("\n");
    }
    else{
        System.out.println("----- PET CANNOT BE FOUND -----");
        System.out.println("\n      It seems that you don't have a Pet, try and ADOPT one! \n");
        System.out.println("-----");
        System.out.println("\n");
    }

    optionPicked = false;
} //view pet end

//INTERACT WITH PET OPTION
while (optionPicked && userChoice == 3) {

    //STOP INTERACTING AFTER REACHING MAIN GOAL
    if(petBoolData[1][currentUser]){
        System.out.println("===== MASTER TRAINER =====");
        System.out.println("\n      You have achieved the pinnacle of pet training \n");
        System.out.println("=====");

        System.out.println("\n");

        optionPicked = false;
        break;
    }

    //INTERACTION WITH PET MENU
    if(petBoolData[0][currentUser] == true){
        System.out.println("----- INTERACTIONS -----");
        System.out.print("1. | TRAIN    - Gain EXP (Energy -2)" + "\n" +
            "2. | PLAY     - Gain EXP & Mood +1 (Energy -1)" + "\n" +
            "3. | FEED     - Restore Energy & Mood" + "\n" +
            "4. | REST     - Restore Big Energy" + "\n" +
            "5. | GO BACK" + "\n");
        System.out.println("-----");
        System.out.print(">>Enter your choice: ");

        if(!sc.hasNextInt()) {
            System.out.println("\n");
            System.out.println("===== NOT A NUMBER =====");
            System.out.println("\n      Invalid input! Please enter a number. \n");
            System.out.println("=====");
            System.out.println("\n");
            sc.next();
            continue;
        }

        interactPetChoice = sc.nextInt();
        sc.nextLine();

        System.out.println("\n");

        petInteract = true;
    }
    else{
        System.out.println("----- PET CANNOT BE FOUND -----");
        System.out.println("\n      You currently have no pet to play with, Try and get one! \n");
        System.out.println("-----");

        System.out.println("\n");
        optionPicked = false;
        break;
    }
} //interaction menu end

//TRAIN OPTION
if(petInteract && interactPetChoice == 1){
    System.out.println("----- PET TRAINING -----");

    if(petIntData[3][currentUser] > 1){
        System.out.println("\n      " + credentials[0][currentUser] + " trained " + petStrData[1][currentUser] +
            "\n      " + petStrData[0][currentUser] + "\n");

        int expGain = (int)(Math.random()*50);
        petIntData[1][currentUser] += expGain;
        petIntData[3][currentUser] -= 2;

        System.out.println("\n      " + petStrData[0][currentUser] + " gained " + expGain + " exp" + "\n");
    }
    else{
        System.out.println("\n      " + petStrData[0][currentUser] + " is too tired to do training" + "\n");
        System.out.println("\n      " + petStrData[0][currentUser] + ", its mood has lowered :( " + "\n");
        petCurrentMood -= 1;

        if (petCurrentMood < 0) petCurrentMood = 0;
        if (petCurrentMood > petDefinitions[1].length - 1) petCurrentMood = petDefinitions[1].length - 1;

        petStrData[3][currentUser] = petDefinitions[1][petCurrentMood];
    }
}

```



```

    }

    System.out.println("-----");

    System.out.println("\n");
} //train pet end

//PLAY OPTION
else if (petInteract && interactPetChoice == 2) {
    System.out.println("----- PLAYING WITH PET -----");

    if (petIntData[3][currentUser] > 0) {
        System.out.println("\n      " + credentials[0][currentUser] + " played with " + petStrData[1][currentUser] + " \n" +
petStrData[0][currentUser] + "\n");
        int expGain = (int)(Math.random()*20);
        petIntData[1][currentUser] += expGain;
        petIntData[3][currentUser] -= 1;
        petCurrentMood += 1;

        System.out.println("      " + petStrData[0][currentUser] + " gained " + expGain + " exp" + "\n");
    }
    else {
        System.out.println("\n      " + petStrData[0][currentUser] + " is too tired to play with " + credentials[0][currentUser]);
        System.out.println("      " + petStrData[0][currentUser] + "'s mood has lowered :( " + "\n");
        petCurrentMood -= 1;
    }

    if (petCurrentMood < 0) petCurrentMood = 0;
    if (petCurrentMood > petDefinitions[1].length - 1) petCurrentMood = petDefinitions[1].length - 1;

    petStrData[3][currentUser] = petDefinitions[1][petCurrentMood];

    System.out.println("-----");

    System.out.println("\n");
} //play pet end

//FEED OPTION
else if (petInteract && interactPetChoice == 3) {
    System.out.println("----- FEEDING PET -----");

    System.out.println("\n      " + credentials[0][currentUser] + " fed " + petStrData[1][currentUser] + " \n" +
petStrData[0][currentUser] + "\n");
    int energyGain = (int)(Math.random()*2) + ((petIntData[4][currentUser] - 10)/3) + 1;
    petIntData[3][currentUser] += energyGain;

    if (petIntData[3][currentUser] > petIntData[4][currentUser]) {
        petIntData[3][currentUser] = petIntData[4][currentUser];
    }

    if (petCurrentMood < petDefinitions[1].length - 1) {
        petCurrentMood += 1;
        petStrData[3][currentUser] = petDefinitions[1][petCurrentMood];
    }
    else {
        petStrData[3][currentUser] = petDefinitions[1][petCurrentMood];
    }

    System.out.println("*****");
    System.out.println("\n      " + petStrData[0][currentUser] + " eats the food happily!");
    System.out.println("      " + petStrData[0][currentUser] + " gained " + energyGain + " energy" + "\n");
    System.out.println("*****");

    System.out.println();

    System.out.println("-----");

    System.out.println("\n");
} //feed pet end

//REST OPTION
else if (petInteract && interactPetChoice == 4) {
    System.out.println("----- RESTING PET -----");

    System.out.println("\n      " + "\n" + petStrData[0][currentUser] + "\n" + " sleeps gently on its bed" + "\n");
    int energyGain = (int)(Math.random()*5) + ((petIntData[4][currentUser] - 10)/2) + 1;
    petIntData[3][currentUser] += energyGain;

    if (petIntData[3][currentUser] > petIntData[4][currentUser])
        petIntData[3][currentUser] = petIntData[4][currentUser];

    System.out.println("*****");
    System.out.println("\n      " + petStrData[0][currentUser] + " gained " + energyGain + " energy" + "\n");
    System.out.println("*****");

    System.out.println();

    System.out.println("-----");

    System.out.println("\n");
} //rest pet end

//GO BACK OPTION
else if (petInteract && interactPetChoice == 5) {
    optionPicked = false;
    petInteract = false;
    break;
} //go back end

```

```

//INVALID NUMBER CHOICE INPUT
else{
    System.out.println("===== INVALID CHOICE INPUT =====");
    System.out.println("\n    INVALID INPUT, PLEASE ENTER A VALID INPUT! \n");
    System.out.println("=====");
    System.out.println("\n");
    continue;
}

//AFTER INTERACTIONS
if(petBoolData[0][currentUser] == true){
    //LEVEL UP CHECKER
    if(petIntData[1][currentUser] >= petIntData[2][currentUser]){
        petIntData[1][currentUser] -= petIntData[2][currentUser];
        petIntData[0][currentUser]++;

        petIntData[2][currentUser] = 100 + (petIntData[0][currentUser] - 1) * 25;

        petIntData[4][currentUser]++;
        petIntData[3][currentUser] = 10 + (petIntData[4][currentUser] - 10);

        System.out.println("***** LEVEL UP *****");
        System.out.println("\n    Congrats! " + petStrData[0][currentUser] + " leveled up! \n");
        System.out.println("*****");

        System.out.println("\n");
    }

    //GOAL CHECKER
    if((petIntData[0][currentUser] == 3 && petCurrentGrowth < 1) ||
        (petIntData[0][currentUser] == 6 && petCurrentGrowth < 2) ||
        (petIntData[0][currentUser] == 9 && petCurrentGrowth < 3) &&
        (petCurrentGrowth < petDefinitions[2].length - 1)) {
        petCurrentGrowth += 1;
        petStrData[2][currentUser] = petDefinitions[2][petCurrentGrowth];

        System.out.println("***** PET GROWTH *****");
        System.out.println("\n    " + petStrData[0][currentUser] + "'s growth status is now " + petStrData[2][currentUser] + "\n");
        System.out.println("*****");

        System.out.println("\n");
    }
    else if(petIntData[0][currentUser] >= 10){
        petBoolData[1][currentUser] = true;
        optionPicked = false;
        petInteract = false;
        logIn = false;

        System.out.println("===== GOAL REACHED =====");
        System.out.println("\n" + credentials[0][currentUser] + " has attained \"Master Trainer\" for having " + "\n"
            + petStrData[0][currentUser] + " reached LVL " + petIntData[0][currentUser] + "\n");
        System.out.println("=====");

        System.out.println("\n");
    }

    //DISPLAY
    if(!petBoolData[1][currentUser]){
        int currentEn = petIntData[3][currentUser];
        int capEn = petIntData[4][currentUser];

        System.out.println("===== ACTION RESULT =====" + "\n" +
            "Level          : " + petIntData[0][currentUser] + "\n" +
            "Growth Status    : " + petStrData[2][currentUser] + "\n" +
            "Exp              : " + petIntData[1][currentUser] + "/" + petIntData[2][currentUser] + "\n" +
            "Energy           : " + "[" + "|".repeat(currentEn) + ".repeat(capEn - currentEn) + "]" + " -> " + currentEn + "/"
+ capEn + "\n" +
            "Mood             : " + petStrData[3][currentUser]);
        System.out.println("=====");

        System.out.println("\n");
    }
    //after interactions end
}

petInteract = false;
}

//RELEASE PET OPTION
if(optionPicked && userChoice == 4){
    if(hasPet[currentUser] == true){
        System.out.println("----- RELEASE PET -----");
        System.out.println("\n    You will be releasing your pet " + petName[currentUser] + "\n");
        System.out.println("!!!WARNING: This will delete your current pet!!!");
        System.out.println("-----");
        System.out.println("Enter \"Y\" to proceed");
        System.out.println("Enter \"X\" to cancel");
        System.out.println("-----");
        System.out.print(">>>ENTER CHOICE: ");
        String releaseInput = sc.nextLine();
    }
}

```

```

        System.out.println("\n");

        if(releaseInput.equalsIgnoreCase("v")){
            hasPet[currentUser] = false;

            System.out.println("----- RELEASED PET -----");
            System.out.println(users[currentUser] + " released " + petName[currentUser] + " into the wilderness." + "\n" +
                petName[currentUser] + " will forever remember the one who took great care of him/her.^^");
            System.out.println("-----");
        }
        else if(releaseInput.equalsIgnoreCase("x")) {
            System.out.println("----- CANCELLING RELEASE -----");
            System.out.println("\n          GOING BACK TO GAME MENU \n");
            System.out.println("-----");
        }
        else{
            System.out.println("===== INVALID INPUT =====");
            System.out.println("\n          Invalid Input, Try Again!          \n");
            System.out.println("=====");
            System.out.println("\n");
            continue;
        }
    }
    else{
        System.out.println("----- PET CANNOT BE FOUND -----");
        System.out.println("\n          You don't have a pet to release, adopt one first. \n");
        System.out.println("-----");
    }
}

System.out.println("\n");

optionPicked = false;
} //release pet end

//LOGOUT OPTION
if(optionPicked == 5){
    System.out.println("----- ACCOUNT LOGOUT -----");
    System.out.println("\n          " + users[currentUser] + " LOGGED OUT OF PETSIM RPG... \n");
    System.out.println("-----");
    System.out.println("\n");

    optionPicked = false;
    login = false;
} //logout option end

//OUT OF BOUNDS HANDLING
if(optionPicked == (userChoice < 1 || userChoice > 5)){
    System.out.println("===== INVALID NUMBER CHOICE =====");
    System.out.println("\n          INVALID NUMBER CHOICE, TRY AGAIN!! \n");
    System.out.println("=====");
    System.out.println("\n");

    optionPicked = false;
}

}

} //main menu end

//LAST MESSAGE
System.out.println("=====");
System.out.println("          THANKYOU FOR PLAYING PETSIM RPG!          ");
System.out.println("=====");

sc.close();
}
}

```

CHAPTER II

RESULTS AND DISCUSSIONS

A. Results

PETSIM RPG Main Menu

Figure 18 shows the main menu of the PETSIM RPG system. It presents three core options: SIGNUP, LOGIN, and EXIT, allowing users to create an account, access an existing one, or terminate the session. The system prompts the user to enter their choice to proceed with the selected operation.

```
===== WELCOME TO PETSIM RPG =====
1. | SIGNUP
2. | LOGIN
3. | EXIT
=====
>>YOUR CHOICE: |
```

Figure 18. PETSIM RPG Main Menu

PETSIM RPG Pet Management Menu

Figure 19 displays the main menu presented to authenticated users in the PETSIM RPG system. It offers five core options: ADOPT A PET, VIEW PET, INTERACT WITH THE PET, RELEASE THE PET, and LOGOUT. These options allow users to manage their virtual pet, engage in interactive gameplay, or exit the session.

```
-----***** PetSim RPG *****-----
1. | ADOPT A PET
2. | VIEW PET
3. | INTERACT WITH THE PET
4. | RELEASE THE PET
5. | LOGOUT
=====*****=====
>>YOUR CHOICE: |
```

Figure 19. PETSIM RPG Pet Management Menu

RPG Pet Selection Menu

Figure 20 presents the pet selection interface within the PETSIM RPG system. After choosing to adopt a pet, users are prompted to select from seven options: Cat, Dog, Duck, Pig, Rabbit, Horse, or Exit. This menu initiates the creation of a pet profile based on the user's choice, transitioning the system into the pet management phase.

```
----- CHOOSE YOUR PET -----
1. | Cat
2. | Dog
3. | Duck
4. | Pig
5. | Rabbit
6. | Horse
7. | Exit
-----
>>Your Choice: |
```

Figure 20. PETSIM RPG Pet Selection Menu

PETSIM RPG Pet Status Screen

Figure 21 displays the current status of the user's adopted pet in the PETSIM RPG system. It includes key attributes such as Pet Name, Pet Type, Level, Experience Points, Growth Status, Energy, and Mood. These metrics reflect the pet's development and interaction history, updating in response to user actions like feeding, playing, or leveling up.

```
----- PET INFORMATION -----
Pet Name      : Maruzensky
Pet Type      : Horse
Pet Level     : 1
Pet Exp       : 0/100
Growth Status : Baby
Pet Energy    : [|||||||||] -> 10/10
Current Mood  : Neutral
-----
```

Figure 21. PETSIM RPG Pet Status Screen

PETSIM RPG Interaction Menu

Figure 22 presents the interaction options available to users within the PETSIM RPG system. Players can choose to TRAIN, PLAY, FEED, or REST their pet, each action affecting attributes such as EXP, Energy, and Mood. The menu also includes a GO BACK option to return to the previous screen. This interface supports decision-making and resource management, allowing users to influence pet development through strategic choices.

```
----- INTERACTIONS -----  
1. | TRAIN   - Gain EXP (Energy -2)  
2. | PLAY    - Gain EXP & Mood +1 (Energy -1)  
3. | FEED    - Restore Energy & Mood  
4. | REST    - Restore Big Energy  
5. | GO BACK  
-----  
>>Enter your choice: █
```

Figure 22. PETSIM RPG Interaction Menu

PETSIM RPG Action Result: TRAIN Interaction

Figure 23 shows the updated pet status following a TRAIN interaction. The pet's attributes are adjusted, with EXP increasing and Energy decreasing, while other values such as Level, Growth Status, and Mood are displayed to reflect the changes.

```
----- PET INFORMATION -----  
Pet Owner      : Mike  
Pet Name       : Maruzensky  
Pet Type       : Horse  
Pet Level      : 3  
Pet Exp        : 81/150  
Growth Status  : Child  
Pet Energy     : [||||]      ] -> 5/12  
Current Mood   : Happy  
-----
```

Figure 23. PETSIM RPG Action Result: TRAIN Interaction

PETSIM RPG Action Result: PLAY Interaction

Figure 24 shows the updated pet status following a PLAY interaction. The pet's attributes are adjusted, with EXP increasing, Energy decreasing, and Mood improving. The system reflects these changes by displaying the current Level, Growth Status, EXP, Energy, and Mood.

```
----- PLAYING WITH PET -----  
  
Mike played with Horse "Maruzensky"  
Maruzensky gained 19 exp  
  
-----  
  
===== ACTION RESULT =====  
Level           : 1  
Growth Status   : Baby  
Exp             : 19/100  
Energy          : [|||||] ] -> 9/10  
Mood            : Joyful  
=====
```

Figure 24. PETSIM RPG Action Result: PLAY Interaction

PETSIM RPG Action Result: FEED Interaction

Figure 25 shows the updated pet status following a FEED interaction. The pet's attributes are adjusted, with Energy increasing and Mood improving. The system reflects these changes by displaying the current Level, Growth Status, EXP, Energy, and Mood.

```

----- FEEDING PET -----

Mike fed Horse "Maruzensky"

*****

Maruzensky eats the food happily!
Maruzensky gained 1 energy

*****

-----

===== ACTION RESULT =====
Level      : 1
Growth Status : Baby
Exp        : 52/100
Energy     : [||||||| ] -> 8/10
Mood       : Joyful
=====

```

Figure 25. PETSIM RPG Action Result: FEED Interaction

PETSIM RPG Action Result: REST Interaction

Figure 26 shows the updated pet status following a REST interaction. The pet's attributes are adjusted, with Energy increasing. The system displays the current Level, Growth Status, EXP, Energy, and Mood, reflecting the changes after resting.

```

----- RESTING PET -----

"Maruzensky" sleeps gently on its bed

*****

Maruzensky gained 2 energy

*****

-----

===== ACTION RESULT =====
Level      : 1
Growth Status : Baby
Exp        : 52/100
Energy     : [|||||||] -> 10/10
Mood       : Joyful
=====

```

Figure 26. PETSIM RPG Action Result: REST Interaction

PETSIM RPG Release Pet Confirmation

Figure 27 presents the confirmation interface for the RELEASE option within the PETSIM RPG Pet Management Menu. The system prompts the user to confirm or cancel the release of their current pet. Proceeding with the release removes the pet from the system

and resets the pet state. On the other hand, cancelling makes the user return to the Pet Management Menu.

```

----- RELEASE PET -----

      You will be releasing your pet Maruzensky

!!!WARNING: This will delete your current pet!!!
-----
Enter "Y" to proceed
Enter "X" to cancel
-----
>>ENTER CHOICE: Y

----- RELEASED PET -----
Mike released Maruzensky into the wilderness.
Maruzensky will forever remember the one who took great care of him/her.^
-----

```

Figure 27. PETSIM RPG Release Pet Confirmation

PETSIM RPG Multi-User Profiles & Duplicate User Checker

Figure 28 displays pet data for multiple authenticated users, each with a distinct pet profile. Attributes such as Pet Name, Type, Level, EXP, Energy, Mood, and Growth Status are stored and updated independently per user. Thus, demonstrating the implementation of multi-user functionality using parallel arrays to maintain individualized pet progression. Additionally, a duplicate name checker is present to ensure unique credentials.

<pre> ***** ACCOUNT CREATED ***** #User_1: Mike ***** </pre>	<pre> ***** ACCOUNT CREATED ***** #User_2: Daniel ***** </pre>	<pre> ***** ACCOUNT CREATED ***** #User_3: Prince ***** </pre>
<pre> ----- PET INFORMATION ----- Pet Owner : Mike Pet Name : Maruzensky Pet Type : Horse Pet Level : 3 Pet Exp : 81/150 Growth Status : Child Pet Energy : []] -> 5/12 Current Mood : Happy ----- </pre>	<pre> ----- PET INFORMATION ----- Pet Owner : Daniel Pet Name : Aster Pet Type : Rabbit Pet Level : 2 Pet Exp : 115/125 Growth Status : Baby Pet Energy : []] -> 1/11 Current Mood : Joyful ----- </pre>	<pre> ----- PET INFORMATION ----- Pet Owner : Prince Pet Name : Meisho Doto Pet Type : Horse Pet Level : 2 Pet Exp : 107/125 Growth Status : Baby Pet Energy : []] -> 4/11 Current Mood : Sad ----- </pre>
<pre> ===== SIGN UP ===== Create Username: Mike ----- USERNAME ALREADY EXISTS ----- Mike is already taken, input another one ----- </pre>		

Figure 28. PETSIM RPG Multi-User Profiles & Duplicate User Checker

B. Discussions

During the development of the system, several realizations emerged regarding the importance of readability, organization, and logical flow in programming. One of the primary challenges encountered was the tendency toward overnesting, which complicated the structure and reduced clarity. Ensuring that the code remained readable and organized was essential, as was the need to guarantee that the output consistently reflected the intended design. Another significant challenge involved the development of the game's calculations, which required careful balancing to ensure smooth progression and a meaningful gameplay experience.

To address these issues, specific solutions were applied. Flag-controlled variables were introduced to regulate the flow of the system, thereby reducing unnecessary complexity. Related variables and processes were grouped inside while loops, functioning as modular methods that improved organization and traceability. Input validation was also implemented to handle possible invalid inputs, ensuring that the system responded free from user errors.

From these realizations, important key takeaways were learned about the value of structured logic, modular design, and proactive error handling. The insights gained highlight the necessity of prioritizing readability and organization for future projects, as well as the importance of designing calculations that support consistent and engaging progress. Thus, this emphasizes the need for an effective system that caters to the user experience and easier documentation.

CHAPTER III

CONCLUSION AND RECOMMENDATION

A. Conclusion

Conclusively, the system was developed to provide a meaningful way of managing user interactions with a pet while ensuring structured gameplay. The objectives of implementing CRUD (Create, Read, Update, and Delete) functions were successfully achieved through the integration of careful game mechanics and multi-user features. As a result, the system delivered a unique and engaging experience, allowing users to progress individually and enjoy a sense of adventure. The outcomes show that the project met its intended goals and provided users with both functionality and entertainment, demonstrating the value of organized design and interactive features in system development.

B. Recommendation

Despite the system's success in providing a unique experience to its intended users and meeting its objectives, further improvements can be made to enhance the overall system's usability and efficiency. Thus, following recommendations are suggested:

1. Multi-Pet Feature

The addition of a multi-pet feature would provide users with a more meaningful pet-keeping experience. Allowing multiple pets would open opportunities for expanded gameplay and progression ideas, creating deeper engagement and variety.

2. Fighting System

To remain true to the RPG genre, the inclusion of a fighting system alongside the pet would expand the scope of gameplay beyond pet-keeping. This feature

would introduce new challenges and strategies, offering users a richer and more dynamic experience.

3. Optimized Game Calculations

The current game calculations lack depth in progression and require adjustments to ensure meaningful gameplay. Refining these calculations, such as allowing mood and growth status to affect experience gain, would highlight a stronger game structure and provide balanced progression for users.

4. Structurally Organized Code

The current game system shows deep code-nesting, which makes the code confusing and difficult to read. Exploring solutions to improve the structure of the code is an important step for future development. One approach is to apply Object-Oriented Programming (OOP) concepts, which would allow a more dynamic flow by separating responsibilities into classes and methods. This would not only reduce complexity but also make the system easier to maintain, expand, and debug.

REFERENCES

- [1] P. Sullivan, "The benefits of virtual pet therapy and digital pet apps for mental health," *Quabble*, May 20, 2024.
- [2] Z.-H. Chen, C. Liao, T.-C. Chien, and T.-W. Chan, "Animal companions: Fostering children's effort-making by nurturing virtual pets," *British Journal of Educational Technology*, vol. 42, no. 1, pp. 166–180, Sep. 2009,
- [3] H. Salameh and H. Salameh, "Parental Guide: Benefits of Digital Pets for Children's Development," *Thumbs Up!*, May 01, 2025.
- [4] L. Takahashi, "The educational value of robot vs virtual pets," *Robot Pet Friends*, Apr. 23, 2025.
- [5] T. H. M. Ed. Of, "Emotional Well-Being Games – *ORI Learning*," *Ori Learning*, Aug. 28, 2025.
- [6] G. Lampropoulos and A. Sidiropoulos, "Impact of Gamification on Students' Learning Outcomes and Academic Performance: A Longitudinal Study Comparing Online, Traditional, and Gamified Learning," *Education Sciences*, vol. 14, no. 4, p. 367, 2024.
- [7] Z. G. Celasun and S. Ü. Kaya, "Gamification in Education: Unlocking Engagement and Enhancing Learning Outcomes," *Turkish Online Journal of Educational Technology*, vol. 24, no. 1, pp. 59–63, 2025.
- [8] Admin, "How interactive learning encourages empathy through animal welfare," *Kids World Fun Blog*, Aug. 19, 2024.
- [9] G. User, "Virtual pets: fun, engaging, and good for your wellbeing — LifeLift apps," *LifeLift Apps*, Nov. 08, 2024
- [10] M. Connolly, "Using game mechanics to enhance learning and engagement," *LearningMole*, Sep. 26, 2025.
- [11] "Enhance learning: Top game mechanics for education," *Southern Methodist University*, Sep. 05, 2025.